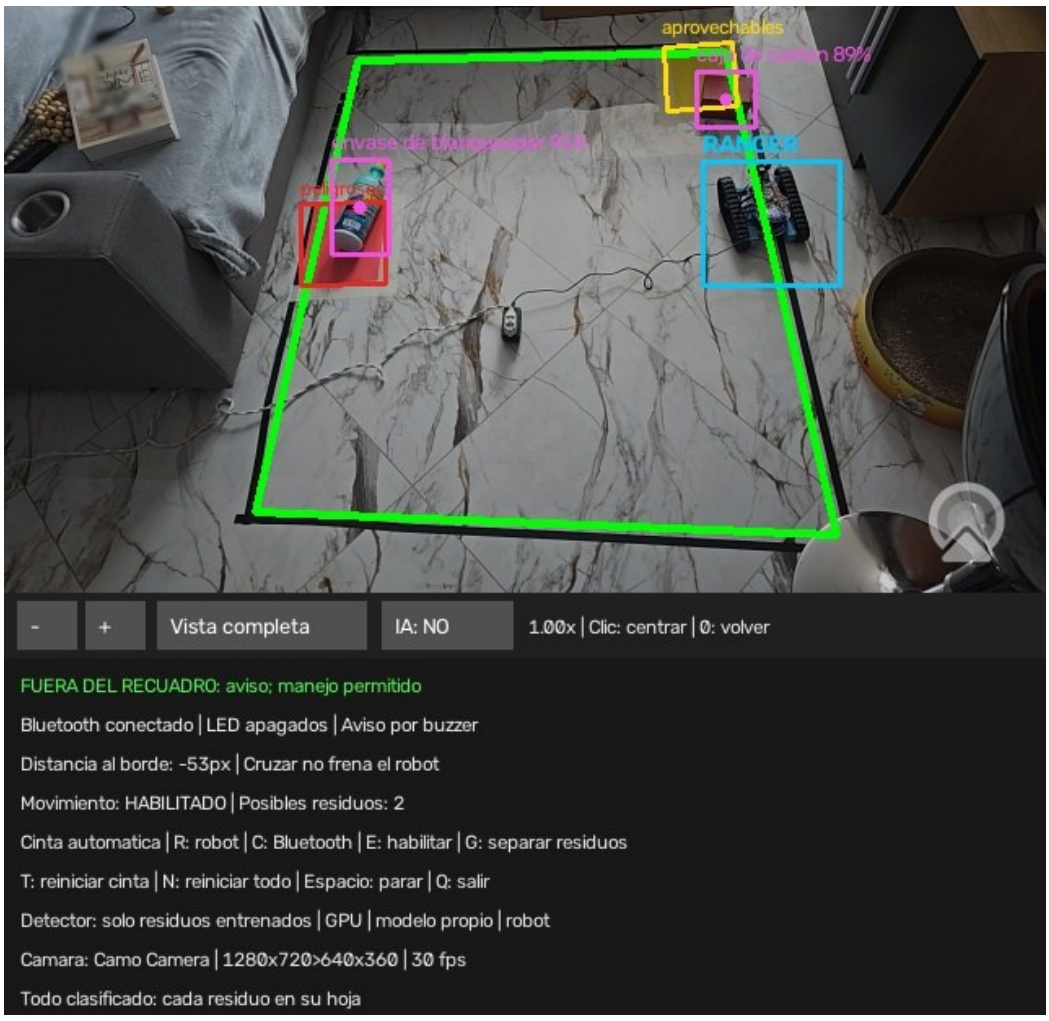
mBot de residuos

Documentación completa



Robot mBot Ranger que reconoce residuos con inteligencia artificial y los lleva a su hoja: **amarilla** para los aprovechables y **roja** para los peligrosos.

Actualizado el 2 de noviembre de 2026

Contenido

Guía de estudio **5**

1. El proyecto en una página . . . 5
2. Componentes . . . 7
3. Recorrido completo, paso a paso . . . 8
4. Seguridad . . . 14
5. Cómo se entrenó la inteligencia artificial . . . 14
6. Historia del proyecto: qué se hizo y por qué . . . 15
7. Resultados medidos . . . 16
8. Limitaciones y trabajo futuro . . . 16
9. Cómo hacer la demostración (lista de comprobación) . . . 17
10. Preguntas que puede hacer el profesor . . . 18
11. Cómo repartir la exposición (propuesta) . . . 19
12. Glosario . . . 19
13. Mapa de archivos . . . 20

Anexo A. Automatización con n8n **21**

Qué hace . . . 21
El flujo . . . 21
Cómo lo usa el programa . . . 21
Instalación (una sola vez) . . . 22
Abrir n8n . . . 22
Configuración (una sola vez, la hace el dueño de la cuenta de Google) . . . 22
Comprobar que funciona . . . 23
Problemas frecuentes . . . 23

Anexo B. Manual de uso **24**

Antes de empezar: el montaje . . . 24
Lanzadores . . . 24
La demostración, paso a paso . . . 25
Qué se ve en la ventana . . . 25
Teclas . . . 27
Mensajes frecuentes . . . 28
El panel web . . . 28
n8n y Google Sheets . . . 29
Manejar el robot a mano . . . 29
Ver residuos sin robot . . . 29
Cámara: LifeCam o Samsung Galaxy . . . 29
Zoom e inspección de objetos . . . 30

Mejora de resolución con IA . . . 30
Diagnóstico . . . 30
Todas las opciones del programa principal . . . 30
Modos antiguos . . . 31
Anexo C. Instalación **32**
Qué necesitas . . . 32
macOS, paso a paso . . . 32
Si falta algún modelo . . . 34
Windows: solo Bluetooth . . . 34
Linux: solo Bluetooth . . . 34
Actualizar o llevar el proyecto a otro Mac . . . 35
Anexo D. Robot, Bluetooth y firmware **36**
El robot . . . 36
Probar la conexión sin moverlo . . . 36
Mover el robot desde la consola (sin cámara) . . . 37
Desde un script de Python . . . 37
Protocolo, para quien programe . . . 38
Control por cable USB (antiguo) . . . 38
Windows y Linux . . . 38
Anexo E. Arquitectura del programa **39**
Palabras que conviene conocer . . . 39
El recorrido de una imagen . . . 39
Archivos de outputs/ . . . 42
Modelos . . . 43
Pruebas automáticas (tests/) . . . 44
Carpetas locales y GitHub . . . 44
Anexo F. Entrenamiento del detector **45**
Qué es entrenar un detector . . . 45
Modelos en uso . . . 45
Cómo se entrenó el detector de cinco residuos . . . 46
Cómo se entrenó el detector del robot . . . 47
Tomar fotos nuevas . . . 47
Etiquetas y volver a entrenar . . . 48
Límites . . . 49
Privacidad . . . 49
Anexo G. Estado del proyecto **50**
En pocas palabras . . . 50

Estado de cada parte . . . 51
Cronología de las pruebas físicas . . . 52
Resultados medidos . . . 53
Lo que no hace (todavía) . . . 53
Problemas abiertos . . . 53
Decisiones que se mantienen . . . 54
Anexo H. Problemas conocidos **55**
Problemas conocidos del prototipo . . . 55
Instalación . . . 55
Cámara . . . 56
Recuadro y hojas . . . 57
Robot . . . 57
Detección . . . 58
Conteo y panel . . . 58
GitHub . . . 59
Qué incluir al pedir ayuda . . . 59
Anexo I. Operación V2 **60**
Firmware V2: qué es y cómo se carga . . . 60
Seguridad: cuándo se para el robot . . . 61
El modo automático (tecla G) . . . 61
Cuándo cuenta un residuo . . . 63
El panel web . . . 64
n8n . . . 64
Comprobaciones antes de una demostración . . . 65

Guía de estudio

Actualizado: 2 de noviembre de 2026. Esta guía es para estudiar el proyecto desde cero y poder exponerlo. Explica qué hace, cómo funciona por dentro paso a paso, qué se hizo y por qué, qué se midió y qué limitaciones tiene. No hace falta saber programar para seguirla: los términos técnicos se explican la primera vez que aparecen y hay un glosario al final.

> **Cómo leerla.** Si tienes poco tiempo, lee el apartado 1 (una página) y el 10 (preguntas del profesor). Para exponer bien, lee del 1 al 9 en orden. El apartado 11 propone cómo repartir la exposición entre varias personas.

1. El proyecto en una página

Qué es. Un prototipo de robot que **separa residuos de forma autónoma**. Una cámara mira una zona del suelo marcada con cinta negra. Dentro hay dos hojas de color y varios residuos. Un programa en el ordenador reconoce cada residuo y al robot, y el robot (un **mBot Ranger** con orugas) empuja cada residuo hasta la hoja que le corresponde:

Hoja	Grupo	Residuos
Amarilla	Aprovechables (reciclables)	caja de cartón, lata, botella de vidrio, botella de plástico
Roja	Peligrosos	envase de blanqueador

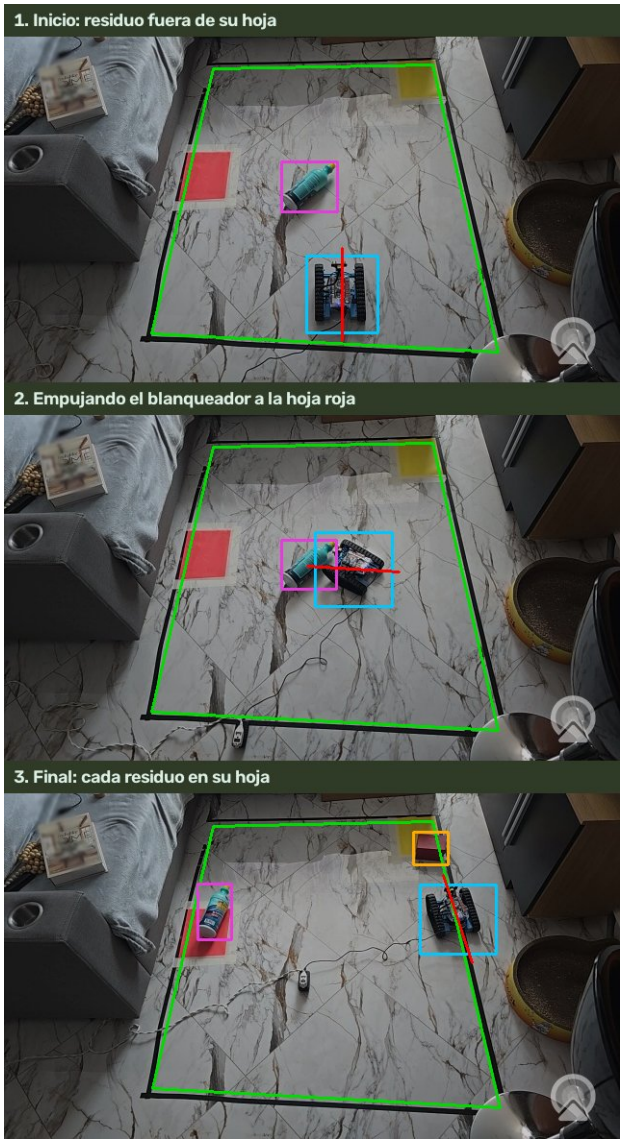
Qué pasa en una demostración, en 5 pasos:

1. La cámara (un teléfono Samsung usado como cámara web) ve la zona. El programa encuentra solo el **recuadro de cinta**, las **dos hojas**, los **residuos** y el **robot**.
2. Se pulsa **G** en el programa. El robot elige el residuo más cercano que no está en su hoja.
3. Se coloca detrás del residuo y lo **empuja hasta su hoja**. Si el residuo es peligroso, va a la roja; si es aprovechable, a la amarilla.
4. Cuando el residuo queda encima de su hoja, el **panel web** lo cuenta («Residuos separados correctamente: +1»).
5. Ese residuo se envía a **n8n** (una herramienta de automatización), que lo **apunta en una hoja de cálculo de Google Sheets** y **responde al panel** con un mensaje («✓ Apuntado en Google Sheets: Lata → hoja amarilla»).

El robot repite esto con cada residuo hasta que todos están en su hoja («Todo clasificado»).

Lo más importante que hay que saber explicar:

- La «inteligencia artificial» del proyecto son **detectores entrenados con fotos propias**: uno reconoce los 5 residuos y otro reconoce el robot.
- El robot no tiene GPS ni brújula: **todo se decide con la cámara cenital**, que mira desde arriba. El programa calcula dónde está cada cosa y hacia dónde mira el robot (por la dirección de sus orugas).
- El ordenador le manda al robot **órdenes cortas por Bluetooth** (avanza un poco, gira un poco, para), y vuelve a mirar después de cada una.
- La **seguridad** está por capas: si algo falla (se pierde la imagen, el recuadro o el robot), el robot se para.



Secuencia real de una separación: el robot lleva el envase de blanqueador a la hoja roja y deja la caja en la amarilla

Imagen real de la prueba del 2 de noviembre. Verde: recuadro de cinta. Azul: robot. Línea roja: dirección de sus orugas. Rosa y naranja: residuos.

2. Componentes

2.1 Hardware (lo físico)

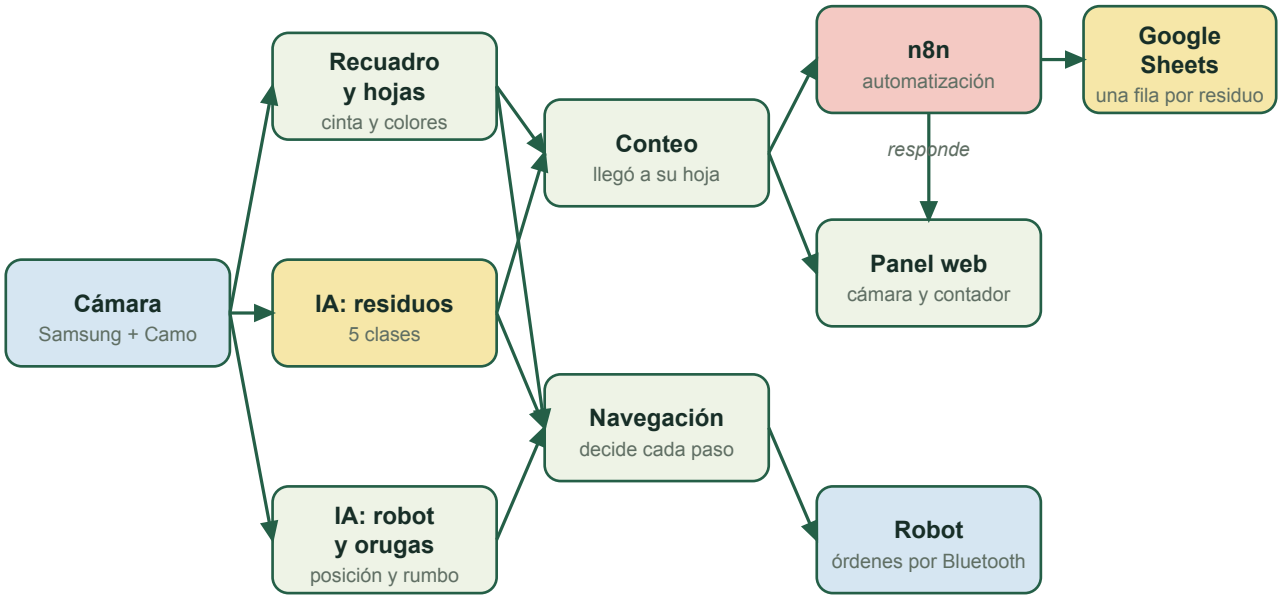
Componente	Para qué sirve
mBot Ranger (placa Auriga, chip ATmega2560)	Robot con orugas que empuja los residuos. Recibe órdenes por Bluetooth.
Samsung Galaxy S24 Ultra con la app **Camo**	Cámara cenital: mira la zona desde arriba y envía la imagen al Mac (1280 × 720, unos 30 fotogramas por segundo).
MacBook con chip Apple M5	Ejecuta todo el programa: visión, inteligencia artificial, navegación, panel y n8n. Usa la tarjeta gráfica (GPU) para la IA.
Cinta negra en el suelo	Marca la zona de trabajo (el «recuadro»).
Hoja amarilla y hoja roja	Destinos de los residuos: aprovechables y peligrosos.
Residuos de prueba	Caja de cartón roja, lata metálica, botella de vidrio pequeña, botella de plástico de agua y envase de blanqueador.

2.2 Software (los programas)

Pieza	Qué es	Dónde está
Programa principal	Une todo: cámara, visión, robot, conteo y panel	`outputs/vision_linea_robot.py`
Python 3.13	Lenguaje en el que está escrito todo el programa del ordenador	`work/venv/`
OpenCV	Biblioteca de visión por ordenador: leer la cámara, buscar colores, líneas y formas	—
YOLO11n (Ultralytics)	Red neuronal que detecta objetos; la entrenamos con nuestras fotos	`outputs/models/*.pt`
Firmware del robot	Programa que corre dentro del mBot (en lenguaje Arduino) y obedece las letras que llegan por Bluetooth	`outputs/ranger_bluetooth.ino`
Panel web	Página que muestra la cámara en vivo y el conteo	`outputs/dashboard.py`, `outputs/dashboard/index.html`
n8n	Herramienta de automatización con bloques visuales; apunta cada residuo en Google Sheets	`outputs/n8n/`
Pruebas automáticas	149 pruebas que comprueban el programa sin cámara ni robot	`tests/`

3. Recorrido completo, paso a paso

Este es el corazón de la exposición: **qué pasa desde que la cámara toma una imagen hasta que aparece una fila en Google Sheets.** El programa repite este ciclo muchas veces por segundo.



Recorrido de una imagen: de la cámara al robot, al panel y a Google Sheets.

3.1 La cámara

- El teléfono se conecta al Mac con la app **Camo** y el Mac lo ve como una cámara web normal.
- El programa elige solo la cámara externa. **Nunca usa la cámara del Mac**: fue una decisión del proyecto, porque no mira el suelo.
- Cada imagen llega en alta resolución. Para el análisis geométrico (cinta, hojas, posiciones) se usa una **copia reducida de 640 píxeles de ancho**, que es más rápida. Para la inteligencia artificial se usa la imagen original, con más detalle.
- Si la imagen deja de cambiar (por ejemplo, el teléfono se queda sin batería y Camo muestra su logo), el programa lo detecta y **para el robot**.

3.2 El recuadro de cinta

Problema: el programa tiene que saber dónde está la zona de trabajo, aunque haya sombras, poca luz o personas alrededor.

Cómo lo encuentra (archivo `vision_cinta.py`):

1. Busca píxeles **más oscuros que el suelo de alrededor** (no solo «negros»: así funciona también con poca luz).
2. Busca **líneas rectas** largas en esas zonas oscuras (técnica de Hough, explicada en el glosario).
3. Prueba combinaciones de cuatro rectas y se queda con las que forman un **cuadrilátero creíble**: esquinas entre 40° y 140°, tamaño razonable, la mayor parte de cada lado con cinta y **suelo claro a los dos lados de la cinta**. Esta última regla descarta los bordes de las sombras.
4. Lo da por **confirmado** cuando lo ve en el mismo sitio durante varias imágenes seguidas.

Resultado medido: en un banco de pruebas con 46 imágenes y 15 tipos de dificultad (oscuridad, sombras, reflejos, zapatos, una persona…), el método antiguo acertaba el **44,9 %** de las veces y el nuevo, el **99,6 %**.

Idea clave: las «coordenadas del recuadro». Como la cámara ve el suelo inclinado, el recuadro aparece deformado (más estrecho por arriba). El programa calcula una transformación matemática (una **homografía**) que «endereza» el recuadro y lo convierte en un cuadrado de 400 × 400 unidades, como un plano visto desde arriba. **Todas las posiciones del robot, los residuos y las hojas se miden en ese plano.** Así las distancias y los ángulos son reales, y si la cámara se mueve un poco, el recuadro se mueve con ella y las cuentas siguen bien.

3.3 Las hojas de color

Cómo las encuentra (archivo `vision_hojas.py`):

- Pasa la imagen al espacio de color **HSV** (tono, saturación, brillo), donde es fácil decir «esto es rojo» o «esto es amarillo».
- Busca manchas grandes, rectangulares y de color intenso dentro del recuadro.
- **Truco para no confundir la hoja roja con la caja roja:** tienen casi el mismo tono, pero la hoja (papel) es **casi tan clara como el suelo o más**, y la caja (cartón con sombra) es **bastante más oscura**.
- Si un residuo tumbado encima **parte la hoja en trozos**, los junta y la reconoce igual, siempre que entre los trozos no haya suelo.
- Una vez confirmadas, las hojas **se recuerdan** en el plano del recuadro, aunque el robot o un residuo las tape un rato.

3.4 Reconocer los residuos (inteligencia artificial)

- Se usa un **detector YOLO11n entrenado por nosotros** con fotos de nuestros 5 residuos (cómo se entrenó: apartado 5).
- Para cada imagen devuelve rectángulos con una etiqueta y una **confianza** (de 0 a 1). Un residuo nuevo solo aparece si la confianza llega a **0,5 en dos imágenes** (o a 0,6 en una). Así se evitan falsas alarmas.
- **Estabilidad:** un residuo no aparece y desaparece en cada imagen. El programa lo sigue en el tiempo y su nombre se decide **por votación** entre las últimas imágenes.
- **Regla de seguridad, «ante la duda, peligroso»:** si el detector duda entre botella de plástico y envase de blanqueador, se trata como **blanqueador**. Es peor mandar un químico al reciclaje que al revés.
- Se descartan las detecciones que caen encima del robot (el robot no es un residuo) y la marca de agua de Camo.

3.5 Reconocer el robot

- Otro **detector propio** (también YOLO11n), entrenado solo para encontrar el robot. En 74 de 74 imágenes de prueba lo encontró (el método anterior, sin entrenar, solo en 1 de 74).
- Entre detección y detección, el robot se **sigue** con un algoritmo llamado **CSRT**, que lo localiza imagen a imagen comparando su aspecto. Si el seguimiento se desvía, el detector lo corrige.
- Antes se probó a pegarle una etiqueta impresa (ArUco), pero se descartó: el robot se reconoce solo.

3.6 Saber hacia dónde mira el robot: las orugas

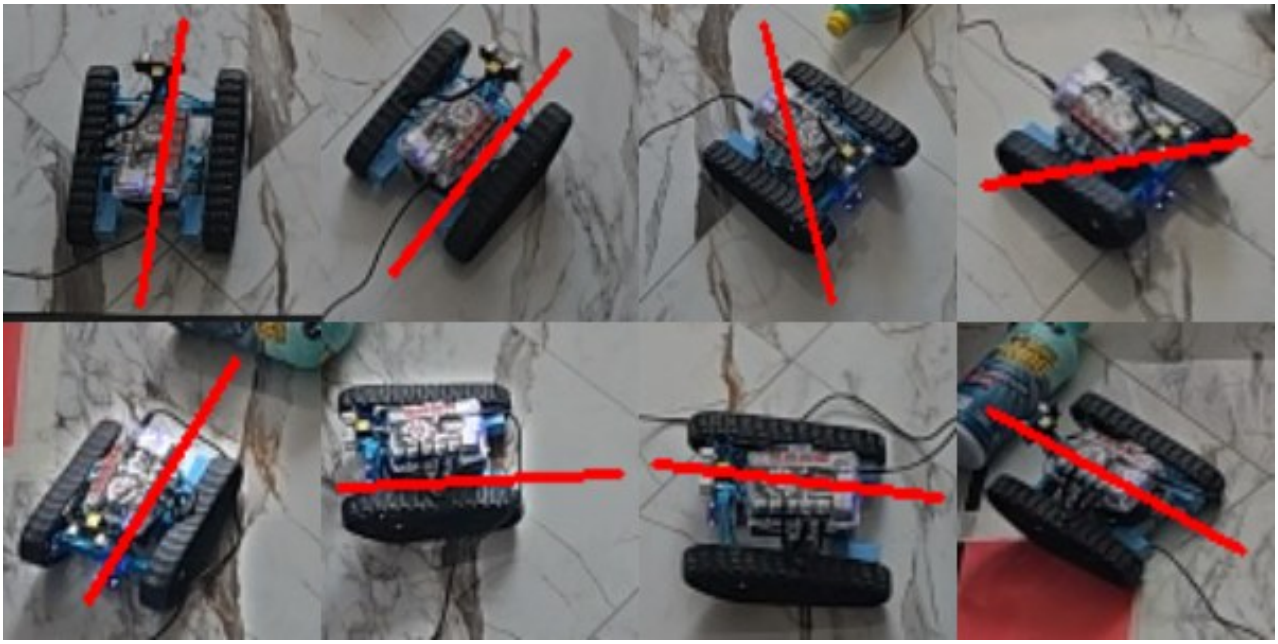
Este fue uno de los problemas más difíciles del proyecto.

El problema: el robot no tiene brújula. Antes, para saber hacia dónde miraba, el programa le hacía dar un **pasito de prueba** después de cada giro y medía hacia dónde se había movido. Pero el rectángulo del robot «tiembla» unos píxeles en la imagen, así que la medida tenía unos **±20° de error**. El siguiente giro se pasaba, había que corregir hacia el otro lado… El robot pasaba **el 60-65 % del tiempo girando o comprobando** y solo un 10-13 % empujando. Era lo que se veía como «vueltas sin sentido».

La solución (archivo `vision_orugas.py`): mirar las **orugas** en la imagen. Son dos franjas negras paralelas a los lados del robot, así que su dirección es la dirección del robot.

1. Se recorta el robot y se marcan sus píxeles oscuros.
2. Se «gira» esa máscara en muchos ángulos y, para cada ángulo, se suma lo oscuro por filas.
3. En la dirección de las orugas aparecen dos picos muy marcados, uno por cada oruga. El ángulo con los picos más marcados es la dirección del robot.

Resultado: comparado con hacia dónde se movió de verdad el robot en 88 tramos rectos grabados, el **error típico es de 5°** (antes, unos 20°). Tarda 4-10 milésimas de segundo. Las orugas dan la dirección pero no el sentido (adelante o atrás), así que el programa elige el sentido más parecido a lo que ya sabía. Si la medida no cuadra con el giro que acaba de hacer, la descarta.



Dirección calculada por las orugas (línea roja) en recortes reales del robot

3.7 La navegación: decidir y mover

Archivo `navegacion.py`. Funciona por **pasos cortos**: decidir, dar una orden corta, esperar a que el robot se quede quieto, mirar dónde quedó y volver a decidir.

1. **Elegir residuo:** el más cercano que esté dentro del recuadro y **no esté ya en su hoja**. Se prefieren los que tienen el camino libre.
2. **Colocarse detrás:** se calcula la línea que va del residuo al centro de su hoja, y el robot se pone detrás del residuo en esa línea. Si el camino pasa por encima del residuo, lo rodea.
3. **Empujar:** avanza hacia el centro de la hoja con el residuo delante, corrigiendo el rumbo con pequeños giros.

4. **Dejarlo:** cuando el residuo está sobre su hoja (su centro encima o al menos el **40 %** de él pisándola), retrocede un poco y vuelve al paso 1.
5. **Terminar:** cuando no quedan residuos fuera de su hoja, da un paso atrás (por si tenía una lata debajo) y muestra **«Todo clasificado»**.

Situaciones especiales que también resuelve:

- **Se sube encima de una lata.** Las orugas trepan por las latas tumbadas. Si el residuo aparece detrás del robot, este retrocede, gira y lo intenta desde otro ángulo.
- **El residuo resbala a un lado** mientras lo empuja: vuelve a colocarse detrás.
- **Se sale del recuadro:** no se para (eso obligaba a ir a buscarlo); vuelve solo hacia el centro.
- **Un residuo imposible:** tras 3 intentos o 2 minutos, lo deja un minuto y sigue con otros.
- **El robot tapa el residuo** al empujarlo: solo lo da por dejado si la última vez que lo vio ya estaba junto a su hoja; si no, retrocede un paso para volver a verlo.

3.8 Las órdenes al robot (Bluetooth y firmware)

- El ordenador habla con el robot por **Bluetooth de baja energía (BLE)**. Le manda **una letra** por orden:

Letra	Orden
`w` / `s`	avanzar / retroceder
`a` / `d`	girar a la izquierda / a la derecha
`x`	parar
`p`	pitido (sin parar los motores)
`0`–`3`	luces (durante la visión van apagadas: alteraban el seguimiento)
`?`	preguntar la versión del firmware

- El robot contesta a cada orden con una confirmación.
- **Temporizador de seguridad en el firmware:** si en **600 milésimas de segundo** no llega otra orden de movimiento, el robot **para solo**. Así, si el ordenador se cuelga o se corta el Bluetooth, el robot no sigue avanzando.
- El firmware se carga en el robot por **USB** con `outputs/actualizar_ranger.py`, y el programa comprueba que la versión sea la correcta antes de mover nada.

3.9 El conteo de residuos separados

Archivo `conteo_residuos.py`. **Cuándo cuenta un residuo:** cuando lo ve **sobre su hoja** habiéndolo visto antes **fuera de ella**. Lo confirma con dos observaciones en al menos 0,35 segundos, o con una sola si nada la contradice durante un segundo.

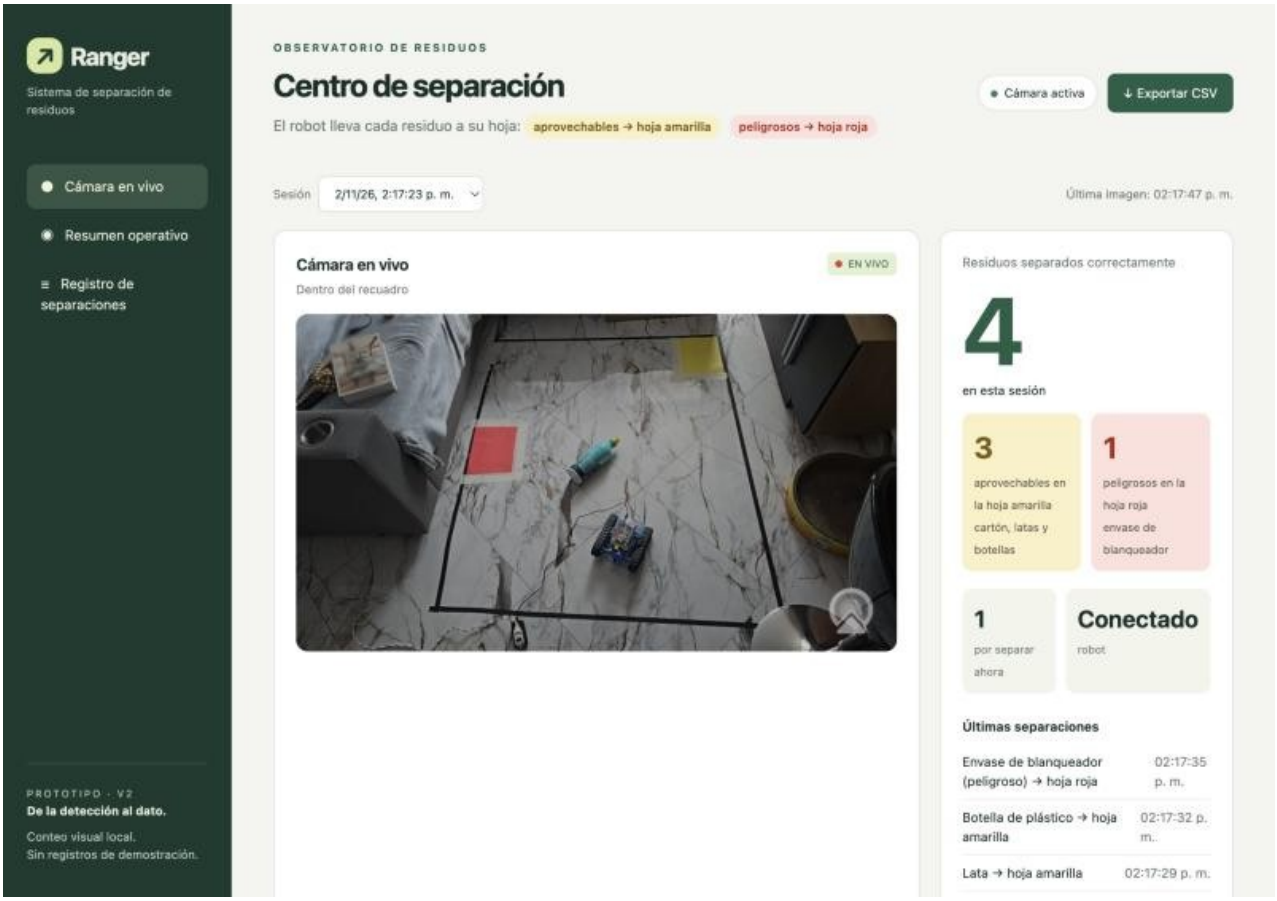
Detalles que evitan errores:

- Las posiciones se comparan en las **coordenadas del recuadro**, así que da igual que la cámara se mueva un poco.
- Cada residuo tiene una «identidad» que se sigue en el tiempo. Cuando el robot lo tapa al empujarlo y reaparece, el programa sabe que es el mismo.
- Si alguien **quita un residuo de su hoja** y lo vuelve a dejar más tarde, **cuenta otra vez**. Si solo «parpadea» en el borde mientras el robot lo empuja, no cuenta doble.

- Si dejas un residuo directamente en su hoja sin que haya estado fuera, no cuenta: no se ha separado.
- Los datos se guardan en una pequeña base de datos local, **SQLite** (`work/estadisticas.sqlite3`).

3.10 El panel web

- Se abre en el navegador en **http://127.0.0.1:8765**. Solo es accesible desde el mismo ordenador.
- Muestra la **cámara en vivo** con las marcas, el **total de residuos separados** en grande, cuántos van a la hoja amarilla y cuántos a la roja, cuántos quedan por separar, si el robot está conectado y las últimas separaciones.
- Al contar un residuo aparece un aviso «+1» sobre la imagen.
- Tiene un apartado **«Automatización n8n»** con las respuestas de n8n.
- Permite **exportar** los datos a CSV y ver sesiones anteriores.

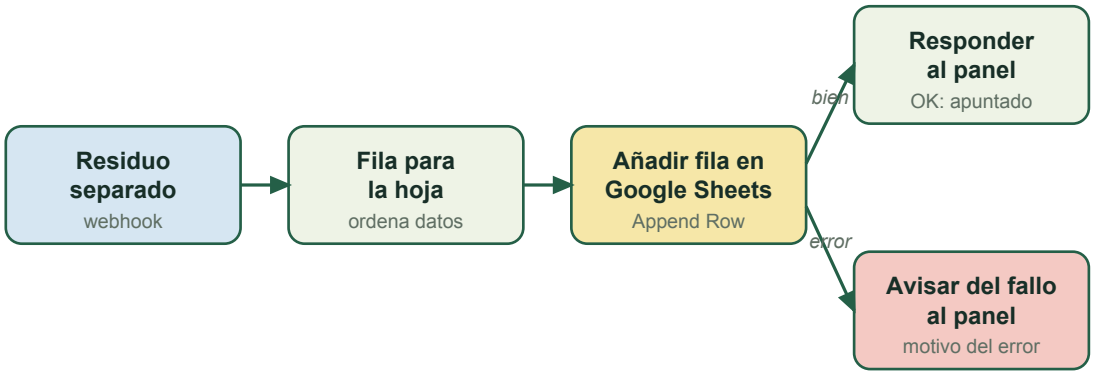


Panel web con la cámara en vivo y el contador (imagen de demostración con una grabación real)

3.11 n8n y Google Sheets

Qué es n8n: una herramienta para crear automatizaciones uniendo **bloques** («nodos») con flechas, sin escribir programas largos. Aquí corre en el mismo Mac (http://localhost:5678).

El flujo (está en `outputs/n8n/registro_residuos.json`) tiene 5 nodos:



El flujo de n8n: recibe el residuo, lo apunta en Google Sheets y responde al panel.

1. **Residuo separado (webhook):** una dirección web que espera avisos. El programa del robot le envía cada residuo contado: fecha, hora, residuo, hoja, grupo, confianza, sesión y total.
2. **Fila para la hoja:** ordena esos datos con los títulos de las columnas.
3. **Añadir fila en Google Sheets:** escribe la fila en la hoja de cálculo «Residuos separados – mBot».
4. **Responder al panel:** contesta al programa «✓ Apuntado en Google Sheets: Lata → hoja amarilla». El panel muestra ese mensaje.
5. **Avisar del fallo al panel:** si Google falla (sin internet, sin permiso…), contesta con el motivo y el panel lo muestra.

Diseño robusto: el envío va «aparte», en otro hilo del programa, así que **el robot nunca espera a internet**. Si n8n está cerrado, los avisos esperan en fila y se mandan cuando se abre. Solo se envían datos del conteo, **nunca imágenes**.



Apartado «Automatización n8n» del panel con las respuestas de n8n

4. Seguridad

El robot se mueve solo, así que la seguridad está pensada por capas:

1. **Parada manual:** la tecla **Espacio** (o `x`) para el robot al instante.
2. **Hay que habilitarlo:** el robot no se mueve hasta que se pulsa **E**, y con **G** empieza el automático.
3. **Imagen reciente:** si la imagen llega con retraso, se congela o desaparece, el robot se para.
4. **Recuadro perdido:** si el programa deja de ver el recuadro, el robot **se deshabilita** y hay que volver a pulsar E y G.
5. **Robot perdido:** si no se ve el robot, el automático se detiene.
6. **Temporizador del firmware:** sin órdenes durante 600 ms, los motores paran solos.
7. **Pasos cortos:** cada orden dura décimas de segundo, nunca un movimiento largo a ciegas.
8. **Bluetooth:** un retraso suelto en la confirmación no corta la conexión, pero tres seguidos sí, y el robot para.

5. Cómo se entrenó la inteligencia artificial

Por qué entrenar un detector propio: los detectores generales, entrenados con millones de fotos de internet, confundían nuestros residuos. Por ejemplo, la caja roja se perdía y la lata se veía solo un 21 % de las veces. Un detector pequeño entrenado con **nuestros objetos, nuestra cámara y nuestro suelo** es mucho más fiable.

Pasos que se siguieron:

1. **Recoger fotos:** con `recoger_imagenes.py`, cientos de fotos desde la cámara de trabajo, con los residuos mezclados, movidos, sobre las hojas, junto al robot y con poca luz.
2. **Preetiquetar:** otros modelos (Grounding DINO y CLIP, que entienden texto como «a turquoise bleach bottle») y reglas de color (turquesa = blanqueador, etiqueta azul = agua) propusieron los rectángulos.
3. **Revisar a mano:** se revisaron todos los recortes en hojas numeradas y se corrigieron unos 70 errores. Con un modelo provisional se encontraron unos 165 residuos sin etiquetar, y 65 blanqueadores más buscando su color turquesa.
4. **Separar fotos de prueba:** una parte de las fotos **no se usa para aprender**, para medir con imágenes que el modelo nunca vio.
5. **Entrenar:** YOLO11n, 35 rondas («épocas»), con 959 fotos para aprender.
6. **Evaluar residuo por residuo** con las fotos de prueba.

Resultados con fotos no vistas (porcentaje de residuos encontrados con la etiqueta correcta):

Residuo	Antes (detector anterior)	Ahora
Caja de cartón	88 %	**100 %**
Lata	23 %	**82 %**
Botella de vidrio	0 %	**72-88 %**
Botella de plástico	0 %	**89 %**
Envase de blanqueador	0 %	**96-99 %**

- **Ninguna confusión** entre botella de plástico y blanqueador en las fotos de prueba.
- Detecciones falsas: 0,04-0,24 por foto (antes 0,54).

- El detector del robot se entrenó igual: 591 fotos, y acertó 74 de 74 en fotos de prueba.

6. Historia del proyecto: qué se hizo y por qué

Fecha (2026)	Qué se hizo	Por qué
Hasta el 19/10	Primera versión: cámara LifeCam, detectores generales de residuos, control manual del robot por Bluetooth, firmware V2 con temporizador de seguridad, primer panel web	Base del proyecto: ver residuos y mover el robot a mano
26/10	Cámara Samsung con Camo; primer detector propio (caja, envoltura, papel)	La LifeCam tenía poca resolución; los detectores generales fallaban
27/10	Firmware V2 cargado en el robot; detector de cinta robusto; robot encontrado sin etiqueta; tecla **G**: primera prueba física: **el robot llegó al residuo y lo sacó del recuadro**	Dar autonomía al robot
28/10	Detector propio del robot; detector propio de caja y lata; panel con cámara en vivo; conteo rehecho; empuje en línea recta; el robot ya no se para al salir del recuadro; se despega si se sube a una lata	Las pruebas físicas mostraron fallos de seguimiento, conteo y vueltas
29/10	4.ª prueba física: 5 residuos sacados y contados	Comprobar los arreglos
1/11	**Hojas de color** como destino; destinos finales: amarilla = aprovechables, roja = peligrosos; detector de **5 residuos**; regla «ante la duda, peligroso»	Pasar de «sacar del recuadro» a **clasificar**
2/11	Pruebas en vivo de las 5 clases; arreglos de conteo y Bluetooth; **dirección por las orugas** (adiós a las vueltas); **n8n + Google Sheets** con respuesta al panel	Dejarlo listo para la exposición

Lecciones que vale la pena contar:

- **Medir antes de cambiar.** Las «vueltas» no se arreglaron a ojo: se midió en las grabaciones que el 60 % del tiempo se iba en girar y comprobar, y se buscó la causa (rumbo mal medido).
- **No todo cambio mejora.** Un ajuste de parámetros que mejoraba en la simulación empeoró en la prueba real y **se deshizo**. Las pruebas físicas mandan.
- **Los datos propios valen más que un modelo grande.** El salto de la lata del 23 % al 82 % vino de etiquetar bien nuestras fotos.

7. Resultados medidos

Qué	Resultado	Cómo se midió
Recuadro de cinta en condiciones difíciles	99,6 % de aciertos (antes 44,9 %)	46 imágenes × 15 tipos de dificultad
Detector de residuos (fotos no vistas)	Caja 100 %, lata 82 %, vidrio 72-88 %, plástico 89 %, blanqueador 96-99 %	80 fotos de prueba
Detector del robot	74 de 74	83 fotos de prueba
Dirección por las orugas	Error típico de 5° (antes ~20°)	88 tramos rectos grabados
Llevar el blanqueador a su hoja	~13 segundos, sin vueltas	Prueba física del 2/11
Navegación en simulación	56 de 60 escenas con los dos residuos en su hoja; 30 s de mediana	60 escenas al azar con ruido y retrasos
n8n	Responde en ~2-2,5 s tras escribir en Google Sheets	Prueba física del 2/11
Pruebas automáticas	149, todas correctas	`python -m unittest discover -s tests`

Ojo al exponer: distingue entre *probado físicamente* (con el robot real), *probado en simulación* (un robot imaginario en el ordenador) y solo *implementado*. Esta guía lo indica siempre.

8. Limitaciones y trabajo futuro

Es un **prototipo**: funciona, pero tiene límites conocidos. Reconocerlos da credibilidad.

- **Residuos pegados a la cinta:** para colocarse detrás, el robot tendría que salir del recuadro y puede perderse de vista. *Mejora posible:* empujar primero el residuo hacia dentro.
- **Todavía gira bastante** para colocarse detrás de cada residuo, aunque mucho menos que antes.
- **La botella de vidrio** se reconoce peor que el resto. Se decidió no seguir mejorándola.
- **Hoja amarilla pequeña:** un residuo puede empujar a otro fuera. En la demostración, se retira cada residuo en cuanto se cuenta.
- **Solo funciona en Mac** (usa la cámara a través de AVFoundation, de Apple).
- **Depende de la luz y del montaje** de la cámara: tiene que ver el recuadro entero y quieto.
- **Empuja, no recoge:** no tiene pinza; recoger con pinza y calcular la ruta óptima serían el siguiente paso.

9. Cómo hacer la demostración (lista de comprobación)

Antes de empezar:

1. Teléfono **cargando**, con Camo abierto y la **cámara trasera**, fijo y mirando el recuadro entero desde arriba.
2. Recuadro de cinta en el suelo; **hoja amarilla y roja dentro**; residuos dentro, **fuera de sus hojas** y separados de la cinta (un largo de robot).
3. Robot **en el suelo**, encendido, con baterías y sin cable tirante.
4. Abrir **n8n**: doble clic en `outputs/n8n/Abrir_n8n.command` y comprobar http://localhost:5678.
5. Abrir el **programa**: doble clic en `outputs/Probar_Robot_Linea.command`.
6. Abrir el **panel** en el navegador: http://127.0.0.1:8765.

Durante la demostración:

1. Comprobar en la ventana del programa que se ven el recuadro verde, las dos hojas, el robot y los residuos.
2. Pulsar **C** (conectar por Bluetooth). Esperar «Bluetooth conectado».
3. Pulsar **E** (habilitar) y **G** (empezar a separar).
4. Tener el dedo en **Espacio** para parar si hace falta.
5. Mostrar en el panel el **+1** y la respuesta de n8n, y en Google Sheets la **fila nueva**.

Si algo falla:

Síntoma	Solución
«Recuadro perdido» y el robot se para	Que nadie tape la cinta; pulsar **E** y **G**
«Bluetooth detenido»	Acercar el robot; pulsar **C**, **E**, **G**
No ve el robot	Que esté entero dentro del recuadro; si no, **R** para marcarlo con el ratón
El robot da vueltas junto a la cinta	Parar con Espacio y separar el residuo de la cinta
El panel dice «n8n sin respuesta»	Abrir n8n; los avisos pendientes se envían solos
Imagen negra con logo de Camo	El teléfono se apagó o Camo se cerró

Teclas del programa: `C` conectar · `E` habilitar · `G` separar residuos · `Espacio`/`X` parar · `W A S D` mover a mano · `R` marcar el robot · `T` reiniciar cinta · `N` reiniciar todo · `B` confirmar recuadro · `+ − 0` zoom · `Q` salir.

10. Preguntas que puede hacer el profesor

¿Dónde está la inteligencia artificial? En dos detectores de objetos (redes neuronales YOLO11n) que **entrenamos con nuestras propias fotos**: uno reconoce los 5 residuos y otro, el robot. Para preparar las fotos también se usaron modelos que entienden texto (Grounding DINO y CLIP). El resto (recuadro, hojas, orugas, navegación) es visión por ordenador clásica y lógica programada.

¿Por qué no usar un detector ya hecho? Porque fallaba con nuestros objetos: la lata se veía un 21-23 % de las veces y la botella de plástico o el blanqueador, nunca. Con fotos propias, el detector pasó a 82-100 % según el residuo.

¿Cómo sabe el robot dónde está y hacia dónde mira? No lo sabe el robot: lo sabe el ordenador mirando desde arriba. La posición sale del detector del robot; la dirección, de sus **orugas** en la imagen. Todo se mide en el plano del recuadro, que se obtiene «enderezando» la imagen con una homografía.

¿Y si confunde el blanqueador con la botella de plástico? Hay una regla: **ante la duda, peligroso**. Si el detector duda entre los dos, lo trata como blanqueador y lo lleva a la hoja roja. En las fotos de prueba no hubo ninguna confusión entre ellos.

¿Qué pasa si se pierde la conexión o la imagen? El robot se para. El firmware tiene un temporizador que detiene los motores si no recibe órdenes en 600 ms, y el programa para el robot si la imagen se atrasa o deja de ver el recuadro o el robot.

¿Para qué sirve n8n? Para **automatizar el registro**: cada residuo separado queda apuntado en Google Sheets con fecha, hora, tipo, hoja y confianza, sin intervención humana. Además n8n responde al panel, que muestra la confirmación. Si n8n está cerrado, el robot sigue funcionando y los avisos se envían después.

¿Cuánto acierta? Ver el apartado 7. Lo más vistoso: el detector de residuos acierta del 82 al 100 % según el residuo en fotos que nunca vio, y el robot llevó el blanqueador a su hoja en unos 13 segundos.

¿Qué no funciona todavía? Ver el apartado 8: residuos pegados a la cinta, todavía algo de giro para colocarse, la botella de vidrio y que solo funciona en Mac.

¿Cómo sabéis que el programa funciona bien? Además de las pruebas físicas, hay **149 pruebas automáticas** que comprueban cada parte sin cámara ni robot. También hay una **simulación** de la navegación con un robot imaginario, que se usó para comparar cambios antes de probarlos con el real.

¿Por qué el robot da pasos cortos y no va directo? Porque cada paso se comprueba con la cámara. Si algo se mueve o el robot resbala, se corrige en el siguiente paso. Es más seguro que un movimiento largo a ciegas.

11. Cómo repartir la exposición (propuesta)

Persona	Parte	Apartados de esta guía
1	El problema, la idea y la demostración	1, 2 y 9
2	Visión: recuadro, hojas y coordenadas del recuadro	3.1, 3.2 y 3.3
3	Inteligencia artificial: residuos, robot y entrenamiento	3.4, 3.5 y 5
4	Navegación, orugas, Bluetooth y seguridad	3.6, 3.7, 3.8 y 4
5	Conteo, panel y automatización con n8n	3.9, 3.10 y 3.11
Todos	Resultados, limitaciones e historia	6, 7 y 8

12. Glosario

- **Algoritmo:** receta de pasos que sigue el programa.
- **BLE (Bluetooth de baja energía):** forma de Bluetooth que usa el robot para recibir órdenes.
- **Confianza:** número de 0 a 1 con el que el detector dice lo seguro que está de lo que ve.
- **CSRT:** algoritmo de seguimiento: localiza un objeto en cada imagen nueva comparándolo con cómo se veía antes.
- **Detector de objetos:** programa de IA que, dada una imagen, devuelve rectángulos con el nombre de cada objeto.
- **Entrenar:** enseñar a una red neuronal con ejemplos (fotos con sus rectángulos y nombres) para que aprenda a reconocerlos.
- **Etiquetar:** marcar en cada foto dónde está cada objeto y qué es.
- **Época:** una vuelta completa del entrenamiento sobre todas las fotos.
- **Firmware:** programa que corre dentro del robot.
- **Fotos de prueba:** fotos que el modelo no ve al aprender; sirven para medir si de verdad generaliza.
- **GPU:** procesador gráfico; hace muy rápido los cálculos de la IA.
- **Homografía:** transformación matemática que convierte la vista inclinada del suelo en un plano visto desde arriba.
- **Hough (transformada de):** técnica para encontrar líneas rectas en una imagen.
- **HSV:** forma de describir colores por tono (qué color), saturación (cuán intenso) y brillo.
- **MJPEG:** vídeo hecho de imágenes JPEG seguidas; así se ve la cámara en el panel.
- **n8n:** herramienta de automatización con bloques visuales (nodos) unidos por flechas.
- **Nodo (en n8n):** cada bloque de una automatización; hace una tarea (recibir, transformar, escribir, responder).
- **OpenCV:** biblioteca de visión por ordenador.
- **Red neuronal:** modelo matemático inspirado en las neuronas que aprende a partir de ejemplos.
- **Simulación:** robot y residuos imaginarios dentro del ordenador para probar la navegación sin riesgo.
- **SQLite:** base de datos sencilla guardada en un archivo.
- **Webhook:** dirección web que espera avisos; cuando llega uno, n8n arranca la automatización.

- **YOLO (You Only Look Once):** familia de detectores de objetos rápidos; usamos la versión pequeña YOLO11n.

13. Mapa de archivos

Archivo	Qué hace
`outputs/vision_linea_robot.py`	Programa principal: cámara, visión, robot, conteo y panel
`outputs/vision_cinta.py`	Encuentra el recuadro de cinta
`outputs/vision_hojas.py`	Encuentra las hojas de color y define qué residuo va a cada una
`outputs/vision_ranger.py`	Cámara, detectores de residuos y del robot, Bluetooth
`outputs/vision_residuos.py`	Estabilidad de las detecciones y regla «ante la duda, peligroso»
`outputs/vision_orugas.py`	Dirección del robot por sus orugas
`outputs/navegacion.py`	Decide los movimientos del robot
`outputs/conteo_residuos.py`	Cuenta los residuos separados y los guarda
`outputs/dashboard.py`, `outputs/dashboard/index.html`	Panel web
`outputs/aviso_n8n.py`	Envía cada residuo a n8n y guarda su respuesta
`outputs/n8n/registro_residuos.json`	El flujo de n8n (se importa en n8n)
`outputs/n8n/Abrir_n8n.command`	Abre n8n en el Mac
`outputs/ranger_bluetooth.ino`	Firmware del robot
`outputs/actualizar_ranger.py`	Carga el firmware en el robot por USB
`outputs/recoger_imagenes.py`, `outputs/entrenar_detector.py`	Recoger fotos y entrenar el detector
`outputs/Probar_Robot_Linea.command`	Abre el programa principal con doble clic
`tests/`	Pruebas automáticas

Para profundizar: Manual de uso · Instalación · Arquitectura · Robot y Bluetooth · Entrenamiento · n8n · Estado · Problemas conocidos

Anexo A. Automatización con n8n

Actualizado: 2 de noviembre de 2026. Si empiezas desde cero, lee primero la guía de estudio (apartado 3.11).

Qué hace

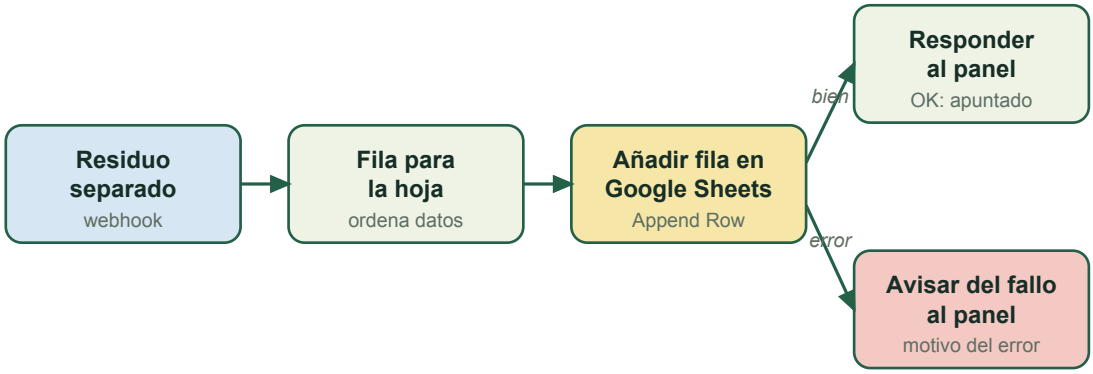
Cada vez que el panel cuenta un residuo separado (un residuo que llega a su hoja de color):

1. El programa del robot lo envía a **n8n**.
2. n8n escribe una **fila** en la hoja de cálculo de Google Sheets «Residuos separados – mBot».
3. n8n **responde** al programa con un mensaje, y el panel lo muestra en el apartado «Automatización n8n», por ejemplo: «✓ Apuntado en Google Sheets: Lata → hoja amarilla».

Columnas de cada fila: **Fecha · Hora · Residuo · Hoja · Grupo · Confianza · Sesión · Total en la sesión**.

El flujo

Archivo: `outputs/n8n/registro_residuos.json`.



El flujo de n8n: recibe el residuo, lo apunta en Google Sheets y responde al panel.

Nodo	Tipo en n8n	Qué hace
Residuo separado	Webhook (POST `/webhook/residuos`, responde con «Respond to Webhook»)	Recibe el aviso del programa
Fila para la hoja	Edit Fields (Set)	Pone los datos con los títulos de las columnas
Añadir fila en Google Sheets	Google Sheets → **Append Row**, **Map Automatically**	Escribe la fila; si falla, sigue por la salida de error
Responder al panel	Respond to Webhook	`{ok: true, mensaje: "✓ Apuntado en Google Sheets: …"}`
Avisar del fallo al panel	Respond to Webhook	`{ok: false, mensaje: "(!) n8n no pudo apuntarlo…: motivo"}`

Cómo lo usa el programa

- Archivo `outputs/aviso_n8n.py`. El envío va en un **hilo aparte**: el robot nunca espera a la red.
- Si n8n está cerrado, los avisos esperan en orden (hasta 500) y se reintentan cada 5 segundos.

- Si n8n contesta con un error, se reintenta como mucho 3 veces y se avisa en el panel. No se reintenta indefinidamente, para no duplicar filas.
- Solo se envían datos del conteo. **Nunca se envían imágenes.**
- Opciones del programa principal: `--n8n URL` (otra dirección de webhook; por defecto `http://localhost:5678/webhook/residuos`) y `--sin-n8n` (desactivarlo).

Instalación (una sola vez)

n8n está instalado **en el Mac**, dentro de `work/n8n`. Esa carpeta no se sube a GitHub. Para instalarlo en otro Mac (hace falta Node.js 24 o más nuevo):

```
mkdir -p work/n8n && cd work/n8n
echo '{"name":"n8n-local","private":true}' > package.json
npm install n8n@2.45.7
```

Abrir n8n

Doble clic en `outputs/n8n/Abrir_n8n.command`. Se abre una ventana de Terminal; no la cierres. Después entra en **http://localhost:5678**. Para cerrar n8n, pulsa Ctrl + C en esa ventana.

El lanzador hace que n8n **solo acepte conexiones desde el mismo Mac**. También quita la «cookie segura», porque Safari no la acepta en `http://localhost`.

Configuración (una sola vez, la hace el dueño de la cuenta de Google)

1. **Usuario de n8n:** en «Set up owner account», escribe correo, nombre y una contraseña (mínimo 8 caracteres, con un número y una mayúscula). Este usuario solo existe en ese Mac.
2. **Importar el flujo:** en n8n, *Create workflow* → *menú* □ → *Import from File* y elige `outputs/n8n/registro_residuos.json`.
3. **Credenciales de Google** en https://console.cloud.google.com:
    1. Crea un proyecto, por ejemplo «mBot residuos».
    2. En *APIs y servicios* → *Biblioteca*, habilita **Google Sheets API** y **Google Drive API**.
    3. En *Pantalla de consentimiento de OAuth*, elige tipo **Externo**, pon un nombre de app y **añade tu correo como usuario de prueba**.
    4. En *Credenciales* → *Crear credenciales* → *ID de cliente de OAuth*, elige tipo **Aplicación web** y como URI de redireccionamiento `http://localhost:5678/rest/oauth2-credential/callback`.
    5. Copia el **ID de cliente** y el **Secreto del cliente**.
4. **Hoja de cálculo:** crea en Google Drive una hoja con esta primera fila: `Fecha | Hora | Residuo | Hoja | Grupo | Confianza | Sesión | Total en la sesión`.
5. **Conectar Google en n8n:** abre el nodo «Añadir fila en Google Sheets». En *Credential*, crea una nueva de tipo «Google Sheets OAuth2 API», pega el ID y el secreto, pulsa *Sign in with Google* y permite el acceso. Si Google avisa de que la app no está verificada, pulsa *Continuar*: la app es tuya.
6. **En el mismo nodo**, comprueba:

  - **Operation: Append Row.** Al elegir la hoja, a veces vuelve sola a «Get Row(s)», que **lee en vez de escribir**: la ejecución sale «correcta» pero no apunta nada.
  - **Document** y **Sheet:** tu hoja.
  - **Mapping Column Mode: Map Automatically.** Con «Map Each Column Manually» y sin valores falla con «At least one value has to be added under 'Values to Send'».
7. **Guardar** (Cmd + S) y pulsar **Publish**, arriba a la derecha. En esta versión de n8n, **cada cambio hay que volver a publicarlo**. Si no, el programa recibe «webhook not registered».

Comprobar que funciona

- En n8n, la pestaña **Executions** muestra cada aviso recibido. En verde quiere decir que se escribió la fila.
- En el panel (http://127.0.0.1:8765), el apartado «Automatización n8n» debe decir **CONECTADO** y mostrar los mensajes.
- Prueba sin robot (desde la carpeta del proyecto, con n8n abierto):

```
curl -s -X POST http://localhost:5678/webhook/residuos -H 'Content-Type: application/json' \
  -d '{"fecha":"2026-11-02","hora":"12:00:00","residuo":"PRUEBA","hoja":"amarilla","grupo":"aprovechables","confianza":0.9,"sesion":"prueba","total_sesion":0}'
```

Debe responder `{"ok":true,"mensaje":"✓ Apuntado en Google Sheets: PRUEBA → hoja amarilla",…}` y aparecer la fila en la hoja.

Problemas frecuentes

Síntoma	Causa	Solución
Safari no abre n8n	Cookie segura en `http://localhost`	Abrir n8n con `Abrir_n8n.command` (ya la desactiva)
«webhook "POST residuos" is not registered»	El flujo no está publicado	Pulsar **Publish**
La ejecución sale bien pero no aparece la fila	El nodo está en «Get Row(s)»	Cambiar a **Append Row**, guardar y publicar
«At least one value has to be added…»	Mapeo manual sin valores	**Map Automatically**, guardar y publicar
El panel dice «SIN RESPUESTA»	n8n cerrado	Abrir n8n; los avisos pendientes se envían solos

Anexo B. Manual de uso

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Instalación · Operación · Problemas

Esta guía explica cómo montar la escena, abrir el programa y hacer que el robot separe los residuos. Todo se hace en el **Mac del proyecto** con la instalación ya hecha (Instalación). Los comandos se escriben en Terminal desde la carpeta del proyecto (la que contiene `README.md` y `outputs`).

Antes de empezar: el montaje

Comprueba esta lista antes de abrir nada:

1. **Recuadro de cinta negra** pegado en un suelo claro. Deben verse sus cuatro lados y sus esquinas.
2. **Dos hojas de papel de color dentro del recuadro**: una **amarilla** (aprovechables) y una **roja** (peligrosos). Planas, sin pliegues y bien visibles.
3. **Residuos dentro del recuadro y fuera de sus hojas**. Déjalos separados de la cinta, al menos a un robot de distancia: si están pegados a ella, el robot tiene que salir del recuadro para colocarse detrás y puede perderse de vista (ver Problemas). Los residuos que conoce el programa son: caja de cartón, lata, botella de plástico y envase de blanqueador (la botella de vidrio casi no se detecta).
4. **El robot dentro del recuadro**, encendido, con el firmware V2 cargado (Operación) y con batería. Sin cables que tiren de él.
5. **El teléfono fijo** encima de la zona, en horizontal, mirando hacia abajo con la cámara trasera. Debe ver el recuadro entero y una franja de suelo alrededor. **No lo sujetes con la mano**: con la imagen movida el recuadro no se confirma.
6. **Luz** uniforme, sin sombras fuertes ni reflejos sobre la cinta.
7. **Nada importante en la esquina inferior derecha** de la imagen: ahí está el logo de Camo y el programa ignora lo que haya.

Lanzadores

Son archivos `.command` de la carpeta `outputs/`. Se abren con doble clic en Finder o desde Terminal.

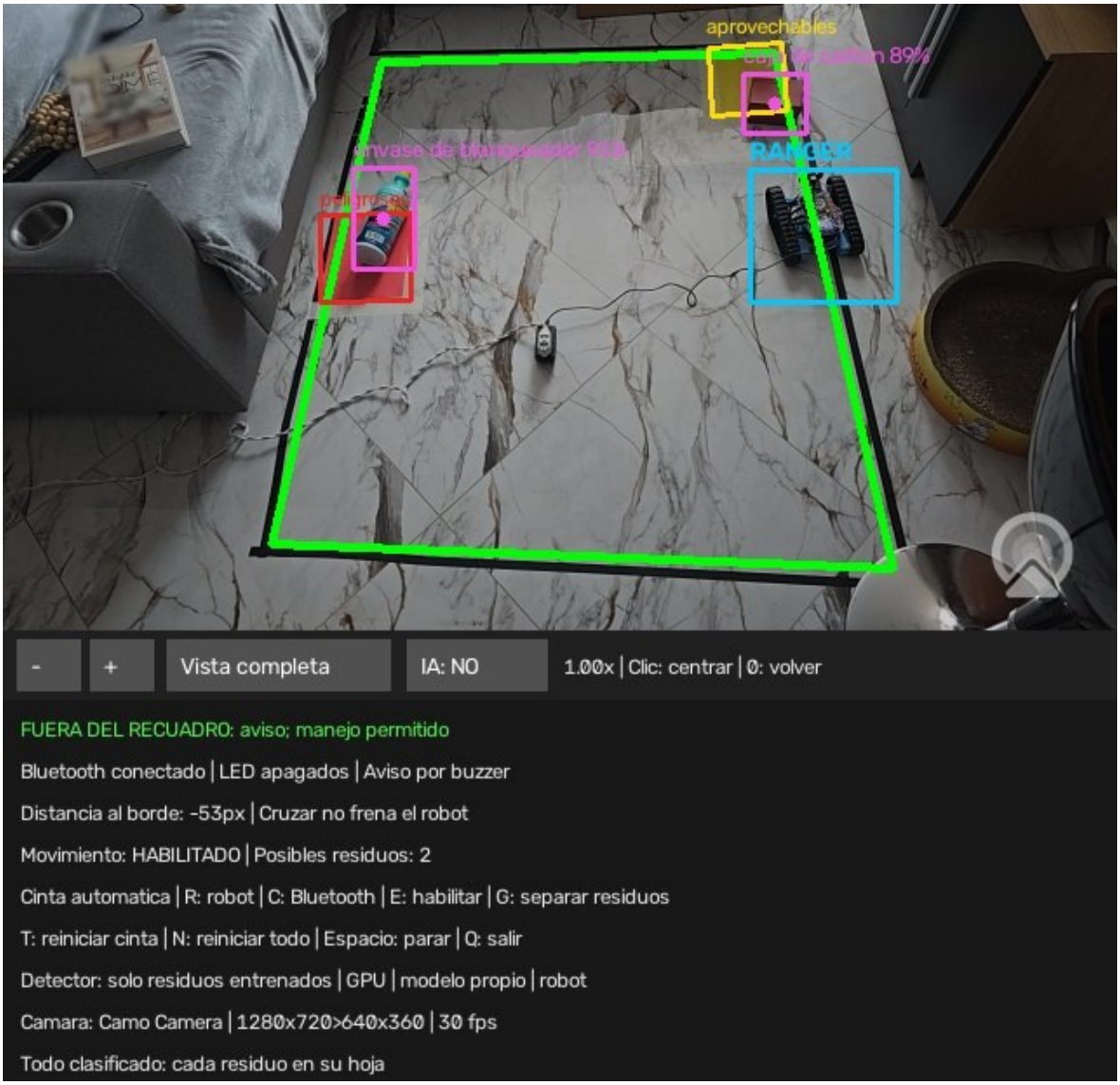
Lanzador	Para qué
`outputs/Probar_Robot_Linea.command`	**El programa principal con el robot.** Es el que se usa en la demostración. Abre también el panel web.
`outputs/n8n/Abrir_n8n.command`	Abre n8n, que apunta cada residuo separado en Google Sheets. Opcional.
`outputs/Abrir_Dashboard.command`	Solo el panel web, sin cámara, para consultar resultados anteriores.
`outputs/Actualizar_Ranger.command`	Carga el firmware en el robot por USB.
`outputs/Recoger_Imagenes.command`	Toma fotos para entrenar el detector (Entrenamiento).
`outputs/Detectar_Residuos.command`	El programa principal sin robot, solo para ver lo que detecta.

La demostración, paso a paso

1. Abre **Camo Studio** en el Mac y la aplicación **Camo** en el teléfono. Comprueba que se ve la imagen de la cámara trasera y que el teléfono tiene batería (o está cargando).
2. (Opcional) Abre `outputs/n8n/Abrir_n8n.command` y deja esa ventana de Terminal abierta. Configurar n8n la primera vez se explica en N8N.md.
3. Abre `outputs/Probar_Robot_Linea.command`. La primera vez tarda un poco: carga los modelos de inteligencia artificial.
4. Espera a que en la ventana del vídeo aparezcan:
   - el **recuadro en verde** (se elige solo cuando se ve estable: al menos 6 imágenes seguidas en 0,4 s);
   - las **hojas** con su color y los rótulos «aprovechables» y «peligrosos» (tardan alrededor de un segundo más);
   - el **robot** dentro de una caja azul con el rótulo «RANGER»;
   - los **residuos** dentro de cajas moradas con su nombre.
5. Abre el panel en el navegador: **http://127.0.0.1:8765**. Debe mostrar la cámara en vivo.
6. Haz clic en la ventana del vídeo (las teclas solo funcionan con ella activa, en minúsculas) y pulsa **C** para conectar el Bluetooth. Espera a leer «Bluetooth conectado | LED apagados | Aviso por buzzer».
7. Pulsa **E** para habilitar el movimiento. La cuarta línea de texto debe decir «Movimiento: HABILITADO».
8. Pulsa **G**. El robot empieza a separar: va a un residuo, se coloca detrás, lo empuja hasta su hoja, retrocede y sigue con el siguiente. La última línea de texto dice qué está haciendo.
9. Cada vez que un residuo queda en su hoja, el panel muestra un aviso «+1» y suma uno. Si n8n está abierto, aparece también su respuesta («Apuntado en Google Sheets…»). **Retira a mano el residuo contado**: la hoja amarilla es pequeña y un residuo empujado podría sacar a otro.
10. Al terminar, el robot pita y la última línea dice «Todo clasificado: cada residuo en su hoja».
11. Para parar en cualquier momento: **Espacio**. Para seguir después: **E** y **G**.
12. Para salir: **Q** en la ventana del vídeo. Cierra n8n con Ctrl+C en su ventana de Terminal.

Si algo se detiene a mitad, mira la tabla de mensajes frecuentes.

Qué se ve en la ventana



Ventana del programa durante una prueba

Dibujos sobre la imagen:

Dibujo	Significado
Cuadrilátero **verde**	Recuadro de cinta confirmado y visible. **Naranja**: se recuerda, pero ahora no se ve bien (el robot no se mueve).
Rectángulos **amarillo** y **rojo** con rótulo	Hojas reconocidas: «aprovechables» (amarilla) y «peligrosos» (roja).
Caja **azul** «RANGER»	El robot que se está siguiendo.
Cajas **moradas** con nombre y porcentaje	Residuos detectados y la confianza del detector. «(seguimiento)» quiere decir que se muestra por su última detección, de hace un momento.
Cajas **amarillas** «objeto por identificar»	Algo dudoso. No se cuenta ni se empuja. Solo aparece con los detectores generales encendidos.
Círculo **celeste** sobre el robot, línea y círculo alrededor del residuo	El plan del modo automático: el robot va hacia ese residuo. Un punto celeste pequeño marca dónde se colocará detrás de él.
Flecha **naranja** desde el residuo	Hacia dónde lo va a empujar (el centro de su hoja).

Dibujo	Significado
Flecha **amarilla** desde el robot	Hacia dónde cree el programa que mira el robot.

Líneas de texto bajo la imagen, de arriba abajo:

1. Estado del recuadro (verde si el robot puede moverse, rojo si no).
2. Estado del Bluetooth.
3. Distancia del robot a la cinta, en píxeles (negativa si está fuera).
4. «Movimiento: HABILITADO» o «BLOQUEADO», y cuántos residuos se ven.
5. y 6. Recordatorio de las teclas.
6. Estado del detector. Lo normal: «Detector: solo residuos entrenados | GPU | modelo propio | robot».
7. Cámara, tamaño de la imagen y su copia de análisis, e imágenes por segundo.
8. Qué está haciendo el modo automático.

Teclas

Tecla	Qué hace
`c`	Conectar el Bluetooth con el robot. Al conectar, para los motores y apaga los LED.
`e`	Habilitar el movimiento. Solo funciona con el recuadro visible, el robot seguido y el Bluetooth conectado.
`g`	Iniciar o detener el modo automático. Necesita lo mismo que `e` y que el movimiento esté habilitado; si falta algo, no arranca.
Espacio o `x`	Parar ya: detiene el robot, deshabilita el movimiento y apaga el modo automático.
`w a s d`	Un paso manual de 0,15 s: adelante, izquierda, atrás, derecha. Apaga el modo automático.
`r`	Elegir el robot con el ratón (respaldo si no se encuentra solo): arrastra un rectángulo que incluya todo el robot, orugas incluidas, y pulsa Enter.
`t`	Borrar el recuadro y las hojas y buscarlos de nuevo. Conserva el robot. Úsalo si moviste la cámara.
`n`	Reiniciar todo: recuadro, hojas y robot.
`b`	Confirmar a mano el recuadro que se ve en amarillo (o una sola línea de cinta).
`+` / `=`, `-`, `0`, `i`	Zoom, vista completa y mejora con IA (ver Zoom).
`q`	Salir (también cerrando la ventana).

`t`, `n`, `b` y `r` detienen el robot y deshabilitan el movimiento: después hay que pulsar `e` (y `g`) otra vez.

Mensajes frecuentes

Mensaje	Qué pasa	Qué hacer
«Lineas detectadas: N - buscando recuadro» o «RECUADRO: estabilizando seleccion automatica...»	Todavía no hay recuadro confirmado.	Espera. Si no llega, comprueba que se ven los cuatro lados, pulsa `t`, o `b` para confirmar el candidato amarillo.
«Borde conservado; esperando cinta visible. Robot detenido.»	La cinta no se ve bien (algo la tapa, cambió la luz, se movió la cámara).	Destápala. Si se movió la cámara, pulsa `t`. Después, **E** y **G**.
«Recuadro perdido o imagen atrasada: detener»	Se perdió el recuadro o la imagen llegó tarde. El robot queda deshabilitado a propósito.	Cuando vuelva el recuadro verde, pulsa **E** y **G**.
«Selecciona TODO el Ranger con R»	No se está siguiendo al robot.	Espera a que el detector lo encuentre, o pulsa `r`. Después, **E** y **G**.
«Bluetooth lento (1/3): se reintenta»	Una confirmación del robot tardó.	Nada; si pasa a «Bluetooth detenido: …», pulsa **C**, **E** y **G**.
«Automatico detenido: robot no habilitado o sin conexion»	Se cortó el Bluetooth o se deshabilitó el movimiento.	**C** (si hace falta), **E** y **G**.
«Automatico detenido: robot no seguido»	Se perdió de vista el robot.	Espera a que vuelva la caja azul (o pulsa `r`), luego **E** y **G**.
«Automatico detenido: el robot no avanza (revisa baterias, orugas o seguimiento)»	Varios pasos sin moverse.	Revisa baterías, que nada lo atasque y que la caja azul esté sobre el robot.
«Automatico: volviendo al recuadro (marcha atras)»	El robot salió del recuadro y vuelve solo.	Nada.
«Automatico: comprobando que no quede nada debajo del robot»	Antes de terminar, retrocede un paso por si tapaba un residuo.	Nada.
«FUERA DEL RECUADRO: aviso; manejo permitido»	El robot está fuera de la cinta. El buzzer pita cada 2 s.	Nada en automático; a mano, vuelve a meterlo.
«Todo clasificado: cada residuo en su hoja»	Terminó: no quedan residuos fuera de su hoja.	Fin.
«Recuadro limpio»	Terminó sacando del recuadro residuos que no tienen hoja.	Fin.
«Automatico: no hay residuos dentro del recuadro»	No ve residuos por separar.	Comprueba que se ven las cajas moradas.

El panel web

Se abre solo con el programa principal en **http://127.0.0.1:8765**. Solo se puede ver desde el mismo Mac. Muestra:

- **Cámara en vivo**, con el recuadro, el robot y los residuos marcados.
- **«Residuos separados correctamente»** en grande, y debajo cuántos hay en la hoja amarilla (aprovechables) y en la roja (peligrosos).
- **«Por separar ahora»**: residuos dentro del recuadro que todavía no están en su hoja. Y si el robot está conectado.
- **Últimas separaciones**, con el aviso «+1 · Lata → hoja amarilla» cada vez que se cuenta uno.
- **Automatización n8n**: CONECTADO, SIN RESPUESTA, APAGADO o PROGRAMA CERRADO, cuántas filas se han escrito en Google Sheets y las últimas respuestas de n8n.

- Indicadores de la sesión, histórico acumulado, gráfico del progreso, reparto por tipo de residuo, registro y botón **Exportar CSV** (para abrir en una hoja de cálculo).

Cada vez que se abre el programa empieza una **sesión nueva** (el contador vuelve a 0); las anteriores se conservan y se eligen en el selector «Sesión». Para poner el panel a cero, cierra el programa y vuelve a abrirlo. Los datos se guardan en `work/estadisticas.sqlite3`: no lo borres.

`Abrir_Dashboard.command` abre solo el panel, sin cámara («Sin imagen de la cámara»), para revisar sesiones anteriores. No hace falta si el programa principal está abierto. Qué cuenta exactamente como «separado» se explica en Operación.

n8n y Google Sheets

Cada residuo contado se envía a n8n (en este mismo Mac, http://localhost:5678), que lo apunta como fila en una hoja de Google Sheets (fecha, hora, residuo, hoja, grupo, confianza, sesión y total) y contesta un mensaje que aparece en el panel. El envío va aparte: si n8n está cerrado, el robot sigue igual y los avisos se reintentan cada 5 s mientras el programa esté abierto. Solo se envían datos del conteo, nunca imágenes.

- Para no enviar nada: `--sin-n8n`.
- Para otra dirección: `--n8n URL` (por defecto `http://localhost:5678/webhook/residuos`).

La instalación y configuración de n8n están en N8N.md.

Manejar el robot a mano

Con el recuadro verde, el robot seguido, **C** y **E**:

- `w`, `a`, `s`, `d` dan pasos de 0,15 s. Mantener la tecla pulsada repite pasos.
- El robot puede acercarse a la cinta y cruzarla: no se frena. Cuando sale del recuadro (más de 5 px durante 0,25 s), el buzzer pita cada 2 s sin parar los motores.
- Espacio o `x` paran.

Un residuo que queda en su hoja cuenta igual, lo empuje el robot en automático, con las teclas o lo coloques con la mano.

Ver residuos sin robot

```
work/venv/bin/python outputs/vision_linea_robot.py --solo-vision
```

Es lo mismo que `outputs/Detectar_Residuos.command`. No conecta el Bluetooth ni busca el robot. Sirve para comprobar la cámara, el recuadro, las hojas y los residuos antes de la demostración. El conteo, el panel y n8n funcionan igual: si colocas a mano un residuo en su hoja, se cuenta. Teclas: `t` (reiniciar recuadro) y `q` (salir).

Cámara: LifeCam o Samsung Galaxy

El programa usa siempre una cámara externa, **nunca la del Mac**. Si hay una sola reconocida (Samsung, Camo o Microsoft LifeCam), la elige sola; si no encuentra ninguna, se detiene y enseña la lista.

Con el **Samsung Galaxy S24 Ultra** se usa **Camo**, porque este teléfono no ofrece la opción «Cámara web» por USB:

1. Instala **Camo** en el teléfono y **Camo Studio** en el Mac (se usó la versión 2.9.0).

2. Activa la extensión de cámara de Camo en el Mac: Ajustes del Sistema → General → Ítems de inicio y extensiones → Extensiones de cámara.
3. Conecta el teléfono por USB (o Wi-Fi), abre Camo y elige la **cámara trasera**.
4. Comprueba qué cámaras ve el Mac:

```
work/venv/bin/python outputs/vision_linea_robot.py --listar-camaras
```

1. Si hay más de una reconocida, elige con `--camara camo`, `--camara samsung`, `--camara lifecam`, parte del nombre o su número en la lista.

Notas:

- Al arrancar, el programa indica el tamaño que entrega de verdad la cámara (por ejemplo 1280 × 720 a 30 imágenes por segundo). El análisis trabaja con una copia de 640 px de lado mayor; los detectores y el zoom usan la original. `--resolucion 1920x1080` pide otro tamaño.
- Si Camo pierde el teléfono, envía imágenes negras con su logo. Si el robot está en marcha y la imagen no cambia en 0,6 s, el programa lo detiene.
- Cuanto más lejos está el teléfono, más pequeños se ven los residuos (en la vista de trabajo miden unos 20-40 px) y peor se detectan. Ponlo lo más cerca posible viendo aún todo el recuadro.
- La LifeCam (640 × 480) sigue funcionando, pero los detectores propios se entrenaron con imágenes del Samsung.

Zoom e inspección de objetos

Solo cambia lo que **ves**; el recuadro, el robot y el conteo siguen usando la imagen completa.

- Botones **+** y **−** o teclas `+`/`=` y `-`: zoom de 1× a 4×.
- Clic sobre un objeto: centra la vista ahí. Clic en la miniatura de la esquina: desplaza.
- **Vista completa** o `0`: vuelve a 1×.
- `--zoom 2` arranca ampliado.

Con el teléfono, el recorte sale de la imagen original, así que muestra detalle real.

Mejora de resolución con IA

El botón **IA** o la tecla `i` activan una mejora ×2 (FSRCNN) sobre la vista ampliada. Es solo para mirar: no cambia lo que detecta el programa. Si tarda más de 150 ms, se apaga sola. Se arranca con `--mejora-ia --zoom 2`. Su modelo (39 KB) lo descarga `preparar_modelos.py`, o solo él con `--solo-superres`.

Diagnóstico

```
work/venv/bin/python outputs/vision_linea_robot.py --diagnostico work/diagnostico
```

Cada segundo guarda, sobrescribiendo, `camara.jpg` (imagen de la cámara), `vista.jpg` (la ventana con los dibujos) y `estado.json` (estado del recuadro, robot, Bluetooth, detecciones, modo automático, n8n, tiempo de cada vuelta y antigüedad de la imagen). Sirve para revisar una prueba después. Guárdalo dentro de `work/`, que no se sube a GitHub: son fotos reales de la casa.

Todas las opciones del programa principal

```
work/venv/bin/python outputs/vision_linea_robot.py --help
```

Opción	Qué hace
`--camara NOMBRE`	Elegir cámara: `camo`, `samsung`, `lifecam`, parte del nombre o número.
`--resolucion ANCHOxALTO`	Pedir un tamaño de captura.
`--listar-camaras`	Mostrar las cámaras del Mac y salir.
`--solo-vision`	Sin robot ni Bluetooth.
`--conectar`	Conectar el Bluetooth al arrancar (no habilita el movimiento).
`--nombre NOMBRE`	Nombre Bluetooth del robot (por defecto `Makeblock_LE10a56219eb91`).
`--diagnostico CARPETA`	Guardar imágenes y estado cada segundo.
`--dashboard-port PUERTO`	Otro puerto para el panel (por defecto 8765).
`--sin-dashboard`	No abrir el panel (el conteo se sigue guardando).
`--n8n URL`	Otra dirección del webhook de n8n.
`--sin-n8n`	No enviar nada a n8n.
`--modelo-propio RUTA` / `--sin-modelo-propio`	Otro detector de residuos entrenado / no usarlo.
`--modelo-robot RUTA` / `--sin-modelo-robot`	Otro detector del robot / buscar el robot solo con Grounding DINO.
`--otros-residuos`	Encender también los detectores generales: más tipos de residuo, muchas más confusiones.
`--modelo RUTA`, `--clases a,b`	Detector general y filtro de sus clases.
`--zoom N`, `--mejora-ia`	Zoom inicial y mejora con IA.
`--velocidad-max`, `--margen`	Se conservan por compatibilidad; ya no frenan al robot.

Modos antiguos

Se conservan por historia; no se usan en la demostración.

- `outputs/vision_linea.py` (`Detectar_Linea.command`): marca líneas negras sueltas, sin robot.
- `outputs/vision_ranger.py` ejecutado directamente: el primer modo, con una etiqueta ArUco impresa (`outputs/Etiqueta_Ranger.png`) pegada encima del robot. El modo actual no la necesita.
- `outputs/ranger_bluetooth.py` sin argumentos: consola Bluetooth sin cámara (Robot).

Anexo C. Instalación

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Uso · Problemas

El proyecto completo (cámara, detección, robot, panel) **solo funciona en Mac**. En Windows y Linux solo se puede usar la consola Bluetooth del robot, y sin probar.

Sistema	Qué funciona
macOS (probado en un Mac con chip Apple M5)	Todo
Windows 11	Solo la consola Bluetooth (Robot). Sin probar con el robot.
Linux	Solo la consola Bluetooth. Sin probar con el robot.

Qué necesitas

Cosa	Para qué
Mac con macOS	Ejecutar el programa. Con chip Apple los detectores van mucho más rápido.
Python 3.13, Git y curl	El programa está en Python. Git hace falta para instalar CLIP; curl, para bajar los modelos.
Unos 3 GB libres	Los modelos generales ocupan algo más de 1 GB; las bibliotecas, otro tanto.
Conexión a Internet	Solo durante la instalación.
Teléfono con Camo (se usó un Samsung Galaxy S24 Ultra), o una webcam Microsoft LifeCam	La cámara. Nunca se usa la del Mac.
mBot Ranger con el firmware V2	El robot (Operación).
Los dos modelos propios, **pedidos a Juan**	Reconocer los cinco residuos y el robot. No están en GitHub.
Cinta aislante negra y dos hojas de papel, amarilla y roja	La escena (Uso).
n8n y una cuenta de Google (opcional)	Apuntar los residuos en Google Sheets (N8N.md).

macOS, paso a paso

Todos los comandos se escriben en **Terminal**, dentro de la carpeta del proyecto (la que contiene `README.md` y `outputs`). Si la ruta tiene espacios, escríbela entre comillas.

1. **Descarga el proyecto** con GitHub Desktop, o con Git:

```
git clone https://github.com/jArango82/mBot_de_residuos.git
cd mBot_de_residuos
```

1. **Comprueba Python, Git y curl.** Si falta alguno, instálalo desde su página oficial.

```
python3.13 --version
git --version
curl --version
```

1. **Crea el entorno de Python** del proyecto (una carpeta `work/venv` con sus propias bibliotecas, sin tocar el resto del Mac) e instala las bibliotecas:

```
python3.13 -m venv work/venv
work/venv/bin/python -m pip install --upgrade pip
work/venv/bin/python -m pip install -r outputs/requirements_vision.txt
```

1. **Descarga los modelos generales** (algo más de 1 GB: YOLO, YOLO-World, CLIP, Grounding DINO y la mejora de imagen). El preparador también genera `residuos-world-small.pt`. No abre la cámara ni conecta el robot.

```
work/venv/bin/python outputs/preparar_modelos.py
work/venv/bin/python outputs/preparar_modelos.py --comprobar
```

`--comprobar` solo mira que los archivos existan; no comprueba que estén bien.

1. **Copia los modelos propios.** Los entrenó Juan con sus fotos y **no están en GitHub**: son archivos pequeños (unos 5,5 MB cada uno), pero el proyecto no sube ningún modelo al repositorio. Pídele estos dos archivos y cópialos en `outputs/models/`:

Archivo	Qué reconoce
`residuos-propio.pt`	Caja de cartón, lata, botella de vidrio, botella de plástico y envase de blanqueador
`robot-propio.pt`	El robot

Sin ellos el programa arranca, pero funciona mucho peor (ver más abajo).

1. **Comprueba OpenCV.** El seguimiento del robot necesita la función CSRT:

```
work/venv/bin/python -c "import cv2; print(cv2.__version__, hasattr(cv2, 'TrackerCSRT_crea
te'))"
```

Debe terminar en `True`. Si no, ver Problemas.

1. **Ejecuta las pruebas automáticas** (unos 20 s, sin cámara ni robot):

```
work/venv/bin/python -m unittest discover -s tests
```

Deben terminar con `Ran 149 tests` y `OK`.

1. **Prepara la cámara.** Instala Camo en el teléfono y Camo Studio en el Mac y activa su extensión de cámara (Uso). Comprueba que el Mac la ve:

```
work/venv/bin/python outputs/vision_linea_robot.py --listar-camaras
```

1. **Primera prueba sin robot:**

```
work/venv/bin/python outputs/vision_linea_robot.py --solo-vision
```

macOS pedirá permiso de **Cámara** (y, con el robot, de **Bluetooth**) para la aplicación desde la que lo abres (Terminal). Acéptalo. La ventana debe mostrar el recuadro y los residuos; el panel, en http://127.0.0.1:8765.

1. **Robot.** Si el robot ya tiene el firmware V2 (el de Juan lo tiene desde el 27/10), no hay que hacer nada. Si no, cárgalo con `outputs/Actualizar_Ranger.command`; necesita **mBlock** instalado en el Mac, porque usa sus herramientas para compilar (Operación). Comprueba la conexión con `work/venv/bin/python outputs/ranger_bluetooth.py --comando p` (debe pitar).

1. **n8n (opcional).** n8n se instala aparte en `work/n8n` (fuera de GitHub) y se abre con `outputs/n8n/Abrir_n8n.command`. Los pasos están en N8N.md. Sin n8n, todo lo demás funciona igual.

Los archivos `.command` son lanzadores de macOS y esperan el entorno en `work/venv`. Si Finder no deja abrirlos, ejecuta en Terminal el comando equivalente de Uso; no hace falta cambiar la seguridad del Mac.

Si falta algún modelo

Falta	Qué pasa
`residuos-propio.pt`	Se usan los detectores generales (YOLO-World, Grounding DINO + CLIP). No distinguen botella de vidrio de plástico, no reconocen el envase de blanqueador y se equivocan mucho más. Una «botella» sin hoja se sacaría del recuadro en vez de llevarla a una hoja.
`robot-propio.pt`	El robot lo busca Grounding DINO cada dos análisis (más de 1 s cada uno, en el procesador). Con el robot pequeño en la imagen falla a menudo; queda `R` para elegirlo con el ratón.
`residuos-world-small.pt` u otro modelo general	El panel de la ventana dice «Detector no disponible»: sin ese archivo no arranca ningún detector, tampoco los propios. Vuelve a ejecutar `preparar_modelos.py`.
`superres/FSRCNN_x2.pb`	Solo falta la mejora de imagen del zoom; todo lo demás funciona.

Windows: solo Bluetooth

Con Windows 11, un adaptador Bluetooth de bajo consumo (BLE) y Python 3.13, en **PowerShell** dentro de la carpeta del proyecto:

```
py -3.13 --version
py -3.13 -m venv work/venv
.\work\venv\Scripts\python.exe -m pip install --upgrade pip
.\work\venv\Scripts\python.exe -m pip install bleak==3.0.2 pyserial==3.5
.\work\venv\Scripts\python.exe outputs/ranger_bluetooth.py --listar
.\work\venv\Scripts\python.exe outputs/ranger_bluetooth.py --comando p
.\work\venv\Scripts\python.exe outputs/ranger_bluetooth.py
```

Activa el Bluetooth de Windows y cierra mBlock u otros programas conectados al robot. Si el robot tiene otro nombre, añade `--nombre "NOMBRE_ENCONTRADO"`. **No ejecutes los programas de cámara en Windows**: usan AVFoundation, que solo existe en Mac.

Linux: solo Bluetooth

Necesitas Python 3.11 o posterior, el módulo `venv`, un adaptador BLE y el servicio BlueZ (5.55 o posterior). Ejemplo en Debian o Ubuntu:

```
sudo apt update
sudo apt install python3 python3-venv bluez
python3 --version
bluetoothctl --version
```

Después, en la carpeta del proyecto y como usuario normal (sin `sudo`):

```
python3 -m venv work/venv
work/venv/bin/python -m pip install --upgrade pip
work/venv/bin/python -m pip install bleak==3.0.2 pyserial==3.5
work/venv/bin/python outputs/ranger_bluetooth.py --listar
work/venv/bin/python outputs/ranger_bluetooth.py --comando p
```

Si se deniega la conexión, revisa los permisos de BlueZ de tu distribución. La cámara no funciona en Linux: habría que adaptar el código de cámara, no basta con instalar OpenCV.

Actualizar o llevar el proyecto a otro Mac

- Para traer cambios: «Pull origin» en GitHub Desktop o `git pull`.
- Si cambió `requirements_vision.txt`, vuelve a ejecutar el paso 3.
- No borres los modelos para actualizar: Git no los toca.
- En otro Mac, crea de nuevo `work/venv` (no se puede copiar de un ordenador a otro), vuelve a descargar los modelos generales y copia los dos modelos propios.
- `work/` no se sube a GitHub: fotos, grabaciones, la base de datos del conteo y n8n se quedan en el Mac donde se crearon. Una copia del repositorio no es una copia de seguridad de esos archivos.

Anexo D. Robot, Bluetooth y firmware

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Operación y firmware · Instalación

Este documento trata del robot por sí solo: su placa, cómo se conecta por Bluetooth, cómo moverlo desde una consola y cómo escribir un script propio. Para usarlo con la cámara, ver el Manual de uso.

El robot

Dato	Valor
Robot	mBot Ranger de Makeblock, montado con orugas
Placa	Auriga (microcontrolador ATmega2560)
Bluetooth	Módulo BLE llamado `Makeblock_LE10a56219eb91`
USB	Chip CH340; en el Mac de Juan aparece como `/dev/cu.usbserial-1120`
Firmware	`outputs/ranger_bluetooth.ino`, versión `RANGER_READY_V2_20261019`, cargado el 27/10

El firmware es nuestro receptor de letras (ver Operación): el programa de fábrica no entiende las órdenes de este proyecto. Si alguien restaura el programa de fábrica, hay que volver a cargarlo con `outputs/Actualizar_Ranger.command`.

Otros archivos de la placa:

- `outputs/ranger-bluetooth.mblock`: el mismo receptor como proyecto de mBlock, por si se quiere cargar desde mBlock (dispositivo mBot Ranger, su puerto y «subir»). La interfaz cambia según la versión de mBlock. El código necesita la biblioteca Makeblock (`MeAuriga.h`): un Arduino IDE sin ella no lo compila.
- `outputs/ranger-original.ino`, `outputs/ranger-original.mblock`: copia del programa anterior.

Después de cargar con mBlock, desconecta mBlock del robot antes de usar Python: dos programas no pueden usar el robot a la vez.

Probar la conexión sin moverlo

Usa el Python del proyecto: `work/venv/bin/python` en Mac y Linux, `.\work\venv\Scripts\python.exe` en Windows (PowerShell). Los ejemplos usan la forma de Mac:

```
work/venv/bin/python outputs/ranger_bluetooth.py --listar
work/venv/bin/python outputs/ranger_bluetooth.py --comando x
work/venv/bin/python outputs/ranger_bluetooth.py --comando p
```

1. `--listar` busca robots cercanos. Debe aparecer `Makeblock_LE10a56219eb91`.
2. `--comando x` envía «parar».
3. `--comando p` hace pitar al robot. Si pita, la conexión y el firmware funcionan.

Si tu robot tiene otro nombre, añade `--nombre "NOMBRE_ENCONTRADO"`.

Al conectar, el programa comprueba que el firmware responde `RANGER_READY_V2_20261019`. Si encuentra una versión antigua, se niega a moverlo y pide conectar el cable USB para actualizarlo. Si hay un cable USB conectado, intenta actualizarlo antes. Necesita las bibliotecas Bleak y pyserial.

Mover el robot desde la consola (sin cámara)

```
work/venv/bin/python outputs/ranger_bluetooth.py
```

Escribe una orden y pulsa Enter:

Orden	Qué hace
`w`, `s`, `a`, `d`	Adelante, atrás, izquierda, derecha durante 0,5 s.
`w 2`	Adelante 2 s. Vale para las cuatro direcciones; entre 0 y 10 s.
`x`	Parar.
`0`, `1`, `2`, `3`	LED apagados, rojos, verdes, azules.
`p`	Pitido de ~200 ms, sin parar los motores.
`q`	Salir (antes pide parar). Esta letra no se envía al robot.

También sirve una orden suelta: `--comando w --segundos 0.5`.

Antes de mover, comprueba que el robot está en el suelo, con batería, con espacio libre y sin cables que tiren de él. **La consola no mira la cámara ni la cinta**: no hay ninguna protección visual, solo el temporizador de 600 ms de la placa.

El movimiento del robot se ha probado con el programa de la cámara (27/10 al 2/11). La consola es una herramienta de prueba.

Desde un script de Python

Guarda tu script en `outputs/`, junto a `ranger_bluetooth.py`, y ejecútalo con el Python del proyecto. Ejemplo que conecta, apaga los LED, pita y para, sin moverse:

```
import asyncio
from ranger_bluetooth import RangerBluetooth

async def main():
    async with RangerBluetooth() as robot:
        await robot.enviar("0")
        await robot.enviar("p")
        await robot.enviar("x")

asyncio.run(main())
```

- `await robot.mover("w", 0.5)` avanza medio segundo: repite la letra cada ~150 ms (para que la placa no se pare a los 600 ms) y envía `x` al terminar.
- `await robot.enviar("w")` envía la letra una sola vez (la placa se parará sola a los 600 ms).
- Para otro robot: `RangerBluetooth(nombre="NOMBRE_ENCONTRADO")`.
- No envíes órdenes desde varias tareas o programas a la vez.

Protocolo, para quien programe

- Escritura (del Mac al robot): característica `0000ffe3-0000-1000-8000-00805f9b34fb`, sin respuesta.
- Avisos (del robot al Mac): característica `0000ffe2-0000-1000-8000-00805f9b34fb`.
- Cada orden es una letra. La placa responde `OK <letra>` y un salto de línea. Ante una letra desconocida, detiene los motores y responde `ERR_COMMAND`.
- `?` devuelve la versión.
- La placa usa el puerto serie a 115200 baudios y potencia fija 100 de 255. No entiende duraciones ni velocidades: los tiempos los decide Python.
- Tras 600 ms sin una letra de movimiento, para y envía `AUTO_STOP`.

El programa de la cámara (`BluetoothGuard` en `outputs/vision_ranger.py`) envía la orden vigente cada 60 ms y espera la confirmación como mucho 0,45 s. Tolera dos confirmaciones lentas seguidas; a la tercera cierra la conexión (antes, una sola la cortaba al empujar).

Control por cable USB (antiguo)

`outputs/ranger.py` y `outputs/ranger_usb.ino` son el primer control por cable. Se conservan, pero no se usan con el firmware actual:

```
work/venv/bin/python outputs/ranger.py --listar
work/venv/bin/python outputs/ranger.py --puerto /dev/cu.PUERTO_REAL --comando p
```

Sustituye el puerto por el que aparezca en la lista. Abrir el puerto puede reiniciar la placa. Nunca abras el USB desde Python y desde mBlock a la vez.

Windows y Linux

En Windows y Linux solo funciona esta parte (Bluetooth y consola), y **no se ha probado** con este robot. Los pasos de instalación están en Instalación. La carga del firmware y todo lo que usa la cámara son solo para Mac.

Anexo E. Arquitectura del programa

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Estado · Uso · Operación

Este documento explica qué piezas tiene el programa, en qué orden trabajan y en qué archivo está cada una. No hace falta saber programar para seguirlo; los términos técnicos se explican la primera vez que aparecen.

Palabras que conviene conocer

Término	Qué es
Detector (YOLO)	Programa de inteligencia artificial que mira una imagen y dibuja un rectángulo («caja») alrededor de cada objeto que conoce, con un nombre y una confianza de 0 a 1. Usamos YOLO11n, un modelo pequeño y rápido, entrenado con fotos nuestras.
Entrenar	Enseñar a un detector con fotos en las que ya se marcó a mano dónde está cada objeto. Ver Entrenamiento.
Seguimiento (CSRT)	Método de OpenCV que, una vez se sabe dónde está el robot, lo sigue de una imagen a la siguiente sin volver a reconocerlo. Es más rápido que el detector, pero se puede despistar.
Coordenadas del recuadro	En vez de medir posiciones en píxeles de la imagen, el programa «aplana» el recuadro de cinta a un cuadrado de 400 × 400 unidades, como si se viera desde arriba. Así, si la cámara se mueve un poco, las posiciones no cambian. La transformación se llama homografía.
Firmware	El pequeño programa que vive en la placa del robot y obedece las letras que le llegan.
BLE	Bluetooth de bajo consumo, por donde el Mac manda las letras al robot.
GPU / MPS	La tarjeta gráfica del Mac (Apple M5). Los detectores van unas seis veces más rápido en ella.
Webhook	Una dirección web que, al recibir un mensaje, pone en marcha un flujo de n8n.

El recorrido de una imagen

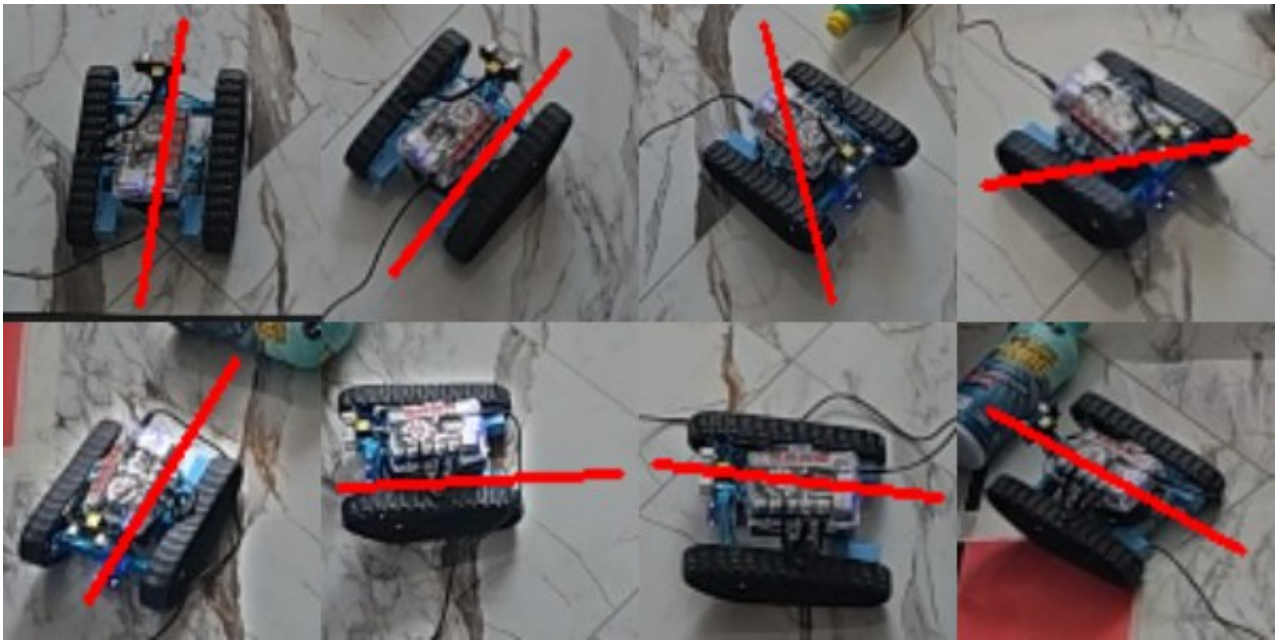
Paso a paso:

1. **Cámara** (`vision_ranger.Camera`, `camaras_mac.py`). Elige sola la única cámara externa reconocida (Samsung, Camo o LifeCam) y rechaza siempre la del Mac. Guarda la imagen original y hace una copia reducida a 640 px de lado mayor (640 × 360 en el teléfono), que es la que usan el recuadro, el seguimiento y el conteo. La original se usa en los detectores y en el zoom. Si el robot está en marcha y la imagen no cambia en 0,6 s (Camo congelado), se detiene.
2. **Recuadro de cinta** (`vision_cinta.py`). Busca zonas más oscuras que el suelo de alrededor, encuentra rectas largas y prueba combinaciones de cuatro que formen un cuadrilátero creíble: cinta en los cuatro lados (aunque algún trozo esté tapado), suelo claro a los dos lados de la cinta (así no confunde sombras) e interior casi sin cinta. Se confirma cuando aparece en el mismo sitio en al menos 6 imágenes durante 0,4 s (`vision_boundary.AutoBoundary`). Con el recuadro ya confirmado, la búsqueda completa se hace una imagen de cada tres y entre medias solo se comprueba que la cinta siga en su sitio.

3. **Hojas de color** (`vision_hojas.py`). Dentro del recuadro busca una mancha grande, rectangular y de color saturado: amarilla, o roja **casi tan clara como el suelo o más** (la caja roja es más oscura y así no se confunde). Se confirma al verla en el mismo sitio 5 veces durante 1 s y se recuerda en coordenadas del recuadro, aunque luego un residuo o el robot la tapen. Si un residuo encima la parte en trozos, también la reconoce.
4. **Detectores** (`vision_ranger.WasteDetector`). En un hilo aparte (un hilo es una tarea que corre a la vez que las demás), el único que usa la gráfica, se ejecutan:
   – el **detector de residuos** propio (`models/residuos-propio.pt`, cinco clases), sobre la imagen original a 800 px;
   – el **detector del robot** propio (`models/robot-propio.pt`), a 640 px.

Los detectores generales (YOLO-World y Grounding DINO + CLIP) quedan apagados porque se equivocaban mucho más; `--otros-residuos` los enciende. Para que un residuo aparezca, el detector propio debe verlo con confianza 0,6 en un análisis, o 0,5 en dos; una vez visto, se mantiene aunque la confianza baje un poco, y su nombre es el más votado en los últimos análisis (así no parpadea). Sobre su propia hoja basta menos confianza, porque el fondo de color lo hace más difícil de ver. Varias reglas evitan errores típicos: si dos clases caen casi en la misma caja y una es blanqueador, gana el blanqueador; una detección que es solo la hoja (la roja parecía una «caja») se descarta; lo que cae sobre el robot no es un residuo.

1. **Robot** (programa principal). Cuando el detector del robot lo ve con confianza 0,5, empieza el seguimiento CSRT. Si el seguimiento se aleja de lo que dice el detector, se corrige; si el aspecto cambia demasiado, se da por perdido y se vuelve a buscar. Sin el modelo del robot, lo busca Grounding DINO (más lento). `R` permite elegirlo a mano.
2. **Orugas** (`vision_orugas.py`). Dentro de la caja del robot busca las dos franjas oscuras paralelas (las orugas) y saca su dirección. Así se sabe hacia dónde mira el robot sin moverlo, aunque no cuál es el frente; eso lo decide la navegación con lo que ya sabe.



Dirección de las orugas detectada en varias imágenes

1. **Navegación** (`navegacion.py`, clase `GoToWaste`). No envía nada por sí misma: en cada imagen decide la siguiente orden («avanza 0,2 s», «gira a la izquierda 0,1 s»…). Elige el residuo más cercano con el camino libre que no esté ya en su hoja, se coloca detrás de él en la línea hacia el centro de su hoja y lo empuja hasta dejarlo encima. Mide hacia dónde mira el robot con las orugas

y con lo que se desplaza tras cada paso, y aprende cuánto gira por segundo. El detalle está en Operación.

2. **Bluetooth** (`vision_ranger.BluetoothGuard`, `ranger_bluetooth.py`). Es el único que habla con el robot. Cada 60 ms mira la orden vigente: si es segura y no ha caducado, envía la letra; si no, envía `x` (parar). Si el robot sale del recuadro, envía `p` (pitido) cada 2 s.
3. **Firmware** (`ranger_bluetooth.ino`). Mueve los motores con cada letra y los para solo si pasan 600 ms sin órdenes nuevas.
4. **Conteo** (`conteo_residuos.py`, clase `SeparationCounter`). Sigue cada residuo en coordenadas del recuadro y apunta un evento cuando queda sobre su hoja habiendo estado antes fuera de ella. Lo guarda en `work/estadisticas.sqlite3`.
5. **Panel web** (`dashboard.py`, `dashboard/index.html`). Un pequeño servidor que solo escucha en el propio Mac lee esa base de datos y la imagen en vivo, y la página se actualiza cada medio segundo.
6. **n8n** (`aviso_n8n.py`, carpeta `n8n/`). Cada evento nuevo se envía, en un hilo aparte, al webhook de n8n, que lo apunta en Google Sheets y contesta un mensaje que se ve en el panel. Si n8n está cerrado, los avisos esperan y se reintentan. Ver N8N.md.

Tareas que corren a la vez

Tarea	Qué hace	Por qué va aparte
Bucle principal	Cámara, recuadro, hojas, seguimiento, navegación, dibujo y teclas, muchas veces por segundo.	Es lo que vigila la seguridad: nunca espera a nada lento.
Detector (gráfica)	Residuos y robot, varias veces por segundo.	Solo un hilo puede usar la gráfica: con dos a la vez el programa se cerraba.
Grounding DINO (procesador)	Solo si falta el modelo del robot o con `--otros-residuos`. ~1,2 s por análisis.	Es lento.
Bluetooth	Envía la orden vigente cada 60 ms.	La radio puede tardar; no debe frenar la imagen.
n8n	Envía los residuos contados.	La red puede tardar o fallar; nunca frena al robot.
Servidor del panel	Atiende al navegador.	Para ver el panel mientras el robot trabaja.

Los modelos se cargan de uno en uno al arrancar: cargarlos a la vez dejaba alguno inservible.

Archivos de `outputs/`

Programa actual

Archivo	Qué contiene
`vision_linea_robot.py`	Programa principal: une todas las piezas, dibuja la ventana y atiende las teclas.
`vision_ranger.py`	Cámara, detectores (`WasteDetector`), estado de las órdenes y el guardián del Bluetooth. Si se ejecuta solo, abre el modo antiguo con etiqueta ArUco.
`camaras_mac.py`	Lista las cámaras del Mac, reconoce Samsung, Camo y LifeCam, y rechaza la del Mac.
`vision_cinta.py`	Detector del recuadro de cinta (desde el 27/10).
`vision_boundary.py`	Confirma el recuadro y decide si sigue visible.
`vision_core.py`	Geometría de líneas y recuadros; también el detector antiguo de recuadro.
`vision_hojas.py`	Hojas de color, qué residuo va a cada hoja (`DESTINO`) y cuándo un residuo está «en su hoja».
`vision_orugas.py`	Dirección del robot por sus orugas.
`vision_residuos.py`	Estabilidad de las detecciones (que no parpadeen) y la regla de precaución plástico/blanqueador.
`vision_papel.py`	Grounding DINO + CLIP: papel, envolturas y, de respaldo, el robot.
`vision_zoom.py`, `vision_superres.py`	Zoom digital y mejora de imagen con IA (solo para mirar de cerca).
`navegacion.py`	Modo automático (tecla G).
`conteo_residuos.py`	Conteo de residuos separados y base de datos.
`dashboard.py`, `dashboard/index.html`	Panel web.
`aviso_n8n.py`, `n8n/registro_residuos.json`	Envío a n8n y el flujo de n8n para importar.
`ranger_bluetooth.py`	Conexión Bluetooth con el robot y consola manual.
`ranger_bluetooth.ino`, `ranger-bluetooth.mblock`	Firmware V2 de la placa y su proyecto de mBlock.
`actualizar_ranger.py`	Compila y carga el firmware por USB.
`recoger_imagenes.py`	Toma fotos para entrenar.
`entrenar_detector.py`	Entrena un detector YOLO11n con las fotos etiquetadas.
`preparar_modelos.py`	Descarga los modelos generales.
`requirements_vision.txt`	Bibliotecas de Python necesarias.

Lanzadores (doble clic en el Mac)

Archivo	Abre
`Probar_Robot_Linea.command`	El programa principal con el robot. **Es el de la demostración.**
`Abrir_Dashboard.command`	Solo el panel web, sin cámara (para consultar el historial).
`Actualizar_Ranger.command`	La carga del firmware por USB.
`Recoger_Imagenes.command`	La toma de fotos para entrenar.
`n8n/Abrir_n8n.command`	n8n en este Mac.
`Detectar_Residuos.command`	El programa principal sin robot (`--solo-vision`).
`Iniciar_Vision.command`	Igual que `Probar_Robot_Linea.command` (nombre antiguo).
`Iniciar_Bluetooth.command`, `Iniciar_Ranger.command`, `Detectar_Linea.command`	Consola Bluetooth, consola USB antigua y prueba de líneas negras.

Histórico (se conserva, no se usa en la demostración)

- `vision_linea.py`: prueba de líneas negras sueltas.
- `vision_ranger.py` ejecutado solo, con `Etiqueta_Ranger.png`: modo antiguo con etiqueta ArUco (un código impreso encima del robot).
- `ranger.py`, `ranger_usb.ino`: control antiguo por cable USB.
- `ranger-original.ino`, `ranger-original.mblock`: copia del programa de la placa anterior.
- `Pendiente_Sensor_Linea.md`: el requisito aplazado del sensor de línea.
- `LEEME_Vision.md`, `LEEME_Bluetooth.md`: notas antiguas que apuntan a estos manuales.

Modelos

Los modelos son archivos grandes y **no están en GitHub** (`.gitignore` los excluye). Van en `outputs/models/`.

Archivo	Qué es	De dónde sale
`residuos-propio.pt`	Detector propio de los cinco residuos (1/11)	Entrenado por Juan: **hay que pedírselo**
`robot-propio.pt`	Detector propio del robot (28/10)	Entrenado por Juan: **hay que pedírselo**
`residuos-world-small.pt`	Detector general YOLO-World con 24 descripciones de residuos	Lo genera `preparar_modelos.py`. El programa lo carga siempre, aunque esté apagado.
`yolov8s-worldv2.pt`, `yolo11n.pt`	Modelos base oficiales	`preparar_modelos.py`
`clip/ViT-B-32.pt`, `grounding-dino-tiny/`	CLIP y Grounding DINO, para papel, envolturas y respaldo del robot	`preparar_modelos.py` (los archivos de configuración de DINO sí están en GitHub; los pesos no)
`superres/FSRCNN_x2.pb`	Mejora de imagen ×2 para el zoom	`preparar_modelos.py`
`residuos-categorias.json`	Las 24 descripciones del detector general	En GitHub

En el Mac de Juan hay además versiones anteriores (`residuos-propio-v1.pt`, `residuos-propio-v3.pt`, `residuos-v2.pt`..., `robot-v0.pt`) que no se usan.

Pruebas automáticas (`tests/`)

149 pruebas que no necesitan cámara ni robot: comprueban con imágenes y situaciones simuladas la elección de cámara, el recuadro, las hojas, la estabilidad de las detecciones, la navegación (con un robot simulado), el conteo, el panel, el envío a n8n, la toma de fotos y el zoom. Se ejecutan con:

```
work/venv/bin/python -m unittest discover -s tests
```

Carpetas locales y GitHub

- `work/` (no se sube): el entorno de Python, las fotos y etiquetas de entrenamiento, las grabaciones de las pruebas, la base de datos del conteo (`work/estadisticas.sqlite3`), la instalación de n8n (`work/n8n`) y las herramientas de etiquetado usadas para entrenar.
- `outputs/models/*.pt` (no se suben): los modelos.
- Todo el análisis de imagen se hace en el Mac. A n8n solo se envían datos del conteo (fecha, hora, residuo, hoja, confianza, sesión), nunca imágenes. La instalación sí necesita Internet para bajar bibliotecas y modelos.

Anexo F. Entrenamiento del detector

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Uso · Estado · Arquitectura

Qué es entrenar un detector

Un **detector** es un programa de inteligencia artificial que mira una foto y dibuja una caja alrededor de cada objeto que conoce, con su nombre y una confianza de 0 a 1. Los detectores generales (como YOLO-World o Grounding DINO) reconocen objetos a partir de una descripción en texto («una lata de aluminio»), pero con nuestros residuos concretos, vistos desde arriba y pequeños, se equivocaban mucho.

Entrenar es enseñar a un detector con ejemplos:

1. Se toman **fotos** de la escena real, con la misma cámara y el mismo montaje que se usarán después.
2. En cada foto se **etiqueta** dónde está cada objeto: un rectángulo y su nombre.
3. Se separan las fotos en dos grupos: unas **para aprender** y otras **para comprobar**, que el modelo nunca ve mientras aprende. Así se sabe si de verdad reconoce los objetos o solo memorizó las fotos.
4. El ordenador ajusta el modelo durante varias pasadas («épocas») sobre las fotos de aprender.
5. Se mide cuántos objetos encuentra en las fotos de comprobar.

Usamos **YOLO11n**, un modelo pequeño y rápido, y lo entrenamos en la gráfica del Mac.

Modelos en uso

Archivo (en)	Qué reconoce	Entrenado	Fotos para aprender
`residuos-propio.pt`	Caja de cartón, lata, botella de vidrio, botella de plástico, envase de blanqueador	1/11	959
`robot-propio.pt`	El robot	28/10	591 (454 con robot y 137 sin él)

Ninguno está en GitHub: hay que pedírselos a Juan (Instalación). El programa los usa si existen; `--sin-modelo-propio` y `--sin-modelo-robot` los desactivan, y `--modelo-propio RUTA` y `--modelo-robot RUTA` permiten probar otros.

Historia

Versión	Fecha	Clases	Resultado
Primer detector	26/10	Caja de cartón, envoltura, papel / servilleta	En una escena fija, las dos cajas y la servilleta se vieron el 100 % del tiempo. Solo conocía esos objetos concretos.
Caja y lata	28/10	Caja de cartón, lata	En 128 fotos no vistas: 96 % de las cajas y 89 % de las latas (con los detectores generales: 25 % y 21 %). Desde entonces los generales se apagan.
Cinco residuos (actual)	1/11	Las cinco clases	Ver abajo.

Cómo se entrenó el detector de cinco residuos

1. **Fotos.** Dos sesiones nuevas el 1/11: una de 145 fotos (casi todas de cerca, solo 17 con la vista de trabajo) y otra de 417 con la cámara fija viendo todo el recuadro: residuos mezclados y movidos, el blanqueador junto a la botella de plástico, cada residuo sobre su hoja, junto al robot y con poca luz. Se sumaron las sesiones anteriores de cajas y latas. **La vista de trabajo es la que importa**: en ella los residuos miden solo 20-40 px.
2. **Etiquetado semiautomático.** Etiquetar a mano cientos de fotos es lento, así que:
   - Grounding DINO y el modelo anterior **propusieron** cajas. Eran muy ruidosas: llamaban «caja» a la hoja roja y confundían el vidrio con plástico o lata.
   - Una limpieza automática clasificó cada propuesta con CLIP (otro modelo que compara imágenes con descripciones) y el color: turquesa = blanqueador, azul = botella de agua, rojo = caja; las hojas, el suelo, las personas y el robot, «no residuo».
   - Después se **revisaron a mano** los recortes, en hojas numeradas: unas 70 correcciones.
   - Un modelo auxiliar entrenado con lo revisado propuso residuos que se habían quedado sin etiqueta (se añadieron unos 165 tras revisarlos), y una búsqueda por color turquesa encontró 65 blanqueadores más.
3. **Entrenamiento.** YOLO11n partiendo del modelo auxiliar, 35 épocas, fotos de 800 px, giros y volteos aleatorios (la cámara mira desde arriba, así que un residuo puede aparecer en cualquier orientación). Unos 70 minutos en la gráfica del Mac.
4. **Comprobación** con fotos que no se usaron para aprender:

Clase	80 fotos de la vista de trabajo	128 fotos de días anteriores
Caja de cartón	100 %	97 %
Lata	82-83 %	94 %
Botella de vidrio	72-88 %	—
Botella de plástico	89 %	—
Envase de blanqueador	96-99 %	—

   - Las dos cifras dependen de cuánto se exige que la caja dibujada coincida con la etiquetada.
   - **Ninguna confusión entre plástico y blanqueador** en esas fotos.
   - Ojo: las 80 fotos de la vista de trabajo son de la misma sesión que parte de las de aprender (otro momento de la misma grabación). La cifra es optimista.

  - Medida estándar del entrenamiento sobre las 208 fotos de prueba (mAP50, de 0 a 1; más es mejor): 0,918 en total; caja 0,994, lata 0,961, vidrio 0,754, plástico 0,959, blanqueador 0,920 (registro `work/entrenar-residuos-v4.log`).

1. **En vivo** (2/11, unos 15 minutos con el robot): lata vista en el 97 % de las imágenes, plástico 89 %, caja 84 %, blanqueador 80 %, **vidrio 5 %**. El vidrio tumbado se confundía con una lata. Juan decidió no seguir con el vidrio.

Además del modelo, el programa añade reglas (Arquitectura): una sola etiqueta por objeto; **si duda entre botella de plástico y blanqueador, elige blanqueador** (peligroso); y sobre su propia hoja acepta menos confianza, porque el fondo de color no aparecía al principio en las fotos.

Cómo se entrenó el detector del robot

Grounding DINO, que buscaba el robot por una descripción, fallaba con el robot pequeño en la imagen (unos 25 px): elegía la pata de una silla, todo el recuadro o nada. Se entrenó un detector propio en dos vueltas:

1. Etiquetas iniciales a partir de las grabaciones de las pruebas (donde se sabía por dónde iba el robot), afinadas con DINO en un recorte ampliado.
2. Un primer modelo (`robot-v0.pt`) propuso cajas en el resto de fotos; se revisaron a mano las dudosas.
3. Modelo final con 591 fotos para aprender (137 sin robot, para que aprenda lo que no es el robot).

Resultado en 83 fotos no vistas de otra prueba: encontró el robot en **74 de 74** y no inventó ninguno en las 9 sin robot, en unos 11 ms por foto. DINO, en esas mismas fotos, lo encontraba en 1 de 74.

Tomar fotos nuevas

Hace falta cerrar antes cualquier programa que use la cámara.

1. Monta el teléfono **fijo**, a la misma altura y con el mismo ángulo que con el robot. Debe verse todo el recuadro y una franja exterior.
2. Abre Camo Studio con el teléfono transmitiendo (cámara trasera).
3. Abre `outputs/Recoger_Imagenes.command`. Empieza en automático: **cada 2 segundos toma una foto y suena un aviso**.
4. Entre aviso y aviso, cambia algo: mueve un residuo, gíralo, quita uno, añade otro.
5. Teclas: `Espacio` guarda una foto ya; `a` activa o desactiva el automático; `q` termina.

No se guardan fotos negras ni congeladas (Camo sin teléfono). En automático tampoco las movidas (`--nitidez-min`, por defecto 60). Con `Espacio` se guardan igual, con un aviso.

Desde Terminal, con nombre de sesión y cada 3 s:

```
work/venv/bin/python outputs/recoger_imagenes.py --sesion tarde-luz-lampara --auto --cada 3
```

Cada vez se crea una **sesión**: una carpeta en `work/dataset/imagenes/` con la fecha y hora, o con el nombre de `--sesion`.

Qué fotografiar

Lo aprendido en estas semanas:

- **La vista de trabajo**, no fotos de cerca: de las 145 fotos de la primera sesión del 1/11, solo 17 tenían la vista de trabajo.

- Cada residuo **en distintas zonas** del recuadro, también junto a la cinta, encima de su hoja y encima de la otra hoja.
- **Girado, tumbado, de pie**, junto a otros, tocándose o tapándose un poco, y **junto al robot**.
- Los residuos que se parecen, **juntos** (botella de plástico y blanqueador).
- **Distintas luces**: de día, de noche, con lámpara, con poca luz.
- Alrededor de **una foto de cada diez sin residuos**, para que aprenda qué no es un residuo (suelo, cinta, hojas, robot).
- Al menos 40-50 fotos de cada residuo, en **varias sesiones** distintas (otro día, otra luz): una sesión entera se reserva para comprobar.
- **Sin personas.** Una mano moviendo un objeto no importa, pero no la etiquetes.

Etiquetas y volver a entrenar

Las etiquetas revisadas de cada sesión se guardan en un archivo JSON, `work/dataset/<carpeta de etiquetas>/<sesión>.json`, con las cajas en píxeles de la foto original:

```
{"classes": ["caja de carton", "lata"],
 "labels": {"0001.jpg": [{"cls": 0, "box": [120, 80, 180, 150]}]}}
```

Los archivos con un punto más en el nombre (`<sesión>.dino.json`, `.modelo.json`, `.limpio.json`) son **propuestas sin revisar** y no se usan para entrenar. Las herramientas de propuesta y revisión que se usaron (`work/etiquetar_cinco.py`, `work/corregir_cinco.py` y otras) están solo en el Mac de Juan, dentro de `work/`, y no en GitHub. **No hay todavía una herramienta cómoda para etiquetar.**

Para entrenar se usa `outputs/entrenar_detector.py`. Ejemplo con los ajustes del detector actual de cinco residuos:

```
work/venv/bin/python outputs/entrenar_detector.py \
  --etiquetas work/dataset/etiquetas-cinco-final \
  --prueba nuevos-vista-b residuos-mov-b empuje-20261027-b \
  --epocas 35 --tam 800 --base outputs/models/residuos5-v0.pt \
  --salida outputs/models/residuos-nuevo.pt
```

Opción	Qué indica
`--prueba`	Sesiones reservadas para comprobar (obligatorio; no se aprende de ellas).
`--etiquetas`	Carpeta de etiquetas (por defecto `work/dataset/etiquetas`).
`--salida`	Dónde guardar el modelo (por defecto `outputs/models/residuos-propio.pt`: ¡sobrescribe el actual!).
`--epocas`	Pasadas de entrenamiento (por defecto 80).
`--tam`	Tamaño de las fotos al entrenar (por defecto 960). El programa analiza luego al mismo tamaño.
`--base`	Modelo de partida (por defecto `yolo11n.pt`).

Para el robot: `--etiquetas work/dataset/etiquetas-robot --prueba empuje-20261027-b --salida outputs/models/robot-propio.pt`.

Prueba el modelo nuevo con `--modelo-propio outputs/models/residuos-nuevo.pt` antes de sustituir el actual, y guarda una copia del anterior. Se guarda el modelo de la última época, no el que mejor puntúa en las fotos de comprobación, para no hacer trampa con ellas. Al terminar, el resultado de la comprobación (mAP50 total y por clase) se imprime y se guarda en

`work/entrenamientos/<nombre>/resultado.json`.

Límites

- Solo reconoce **estos residuos concretos** (la caja roja, la caja blanca y azul, la lata, la botella de plástico, el envase de blanqueador y la botella de vidrio de las fotos). Otra marca u otro tamaño puede no reconocerse.
- **La botella de vidrio** apenas se detecta en vivo, sobre todo tumbada. Se decidió no seguir con ella.
- Se equivoca más con residuos **amontonados** (una lata encima de la caja), **en la mano** o con una luz muy distinta.
- Todo se entrenó en la casa de Juan, con su suelo y su luz: en la exposición puede funcionar peor.

Privacidad

Las fotos se quedan en el Mac, dentro de `work/`, que no se sube a GitHub. Revisa que no aparezcan personas ni documentos antes de compartir alguna. Ningún análisis envía imágenes fuera del Mac.

Anexo G. Estado del proyecto

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Uso · Arquitectura · Problemas

Este documento dice **qué funciona de verdad y cómo lo sabemos**. Usamos tres niveles; al exponer, conviene no mezclarlos:

Nivel	Qué significa
Implementado	El código existe y pasa las pruebas automáticas, pero no se ha visto funcionar con el robot real.
Probado en simulación	Se comprobó con un robot y una cámara simulados en el ordenador, con ruido y retrasos parecidos a los reales. Sirve para encontrar fallos; no demuestra que funcione en el suelo.
Probado físicamente	Se vio funcionar con el robot, la cámara y los residuos reales (en casa de Juan, con luz de día y la cámara fija).

En pocas palabras

Una cámara (un Samsung Galaxy S24 Ultra conectado al Mac con la aplicación Camo) mira desde arriba un recuadro de cinta negra en el suelo. Dentro del recuadro hay dos hojas de papel de color. El programa reconoce cinco tipos de residuo y el robot mBot Ranger. Al pulsar **G**, el robot empuja cada residuo hasta su hoja:

- Hoja **amarilla**, residuos **aprovechables**: caja de cartón, lata, botella de vidrio y botella de plástico.
- Hoja **roja**, residuos **peligrosos**: envase de blanqueador. Si el programa duda entre botella de plástico y blanqueador, lo trata como peligroso.

Un panel web (http://127.0.0.1:8765) muestra la cámara en vivo y cuenta cada residuo que queda sobre su hoja. Cada residuo contado se envía a **n8n**, que lo apunta como fila en una hoja de Google Sheets y responde al panel.

Estado de cada parte

Parte	Estado	Evidencia
Cámara externa (Samsung por Camo; LifeCam también sirve)	Probado físicamente	En uso desde el 26/10. La cámara del Mac se rechaza siempre.
Recuadro de cinta automático	Probado físicamente **con la cámara fija**	Reconocido desde el principio en todas las pruebas del 2/11. Con el teléfono en la mano no llegó a confirmarse (prueba del 27/10).
Hojas amarilla y roja	Probado físicamente	Reconocidas en las pruebas del 2/11, también con un residuo encima.
Detector de los cinco residuos	Probado físicamente, **salvo la botella de vidrio**	Prueba del 2/11 (unos 15 min): lata vista en el 97 % de las imágenes, plástico 89 %, caja 84 %, blanqueador 80 %, vidrio solo 5 %.
Robot reconocido solo, sin etiqueta	Probado físicamente	El detector propio del robot se usó en todas las pruebas del 2/11. Seleccionarlo con el ratón (`R`) queda de respaldo.
Hacia dónde mira el robot, por sus orugas	Probado físicamente	En uso desde el 2/11. Error mediano medido: 5°.
Firmware V2 en la placa del robot	Probado físicamente	Cargado el 27/10. Responde por USB y por Bluetooth.
Llevar cada residuo a su hoja (tecla G)	Probado físicamente	2/11, 14:02: blanqueador a la hoja roja y caja a la amarilla, contados a la primera. Ese día también llegaron a la amarilla la botella de plástico y la lata.
Sacar del recuadro un residuo sin hoja	Probado físicamente con una versión anterior	27-29/10, antes de que existieran las hojas. Hoy solo se usa si un residuo no tiene hoja asignada.
Conteo y panel web con cámara en vivo	Probado físicamente	Contó en las pruebas del 2/11 (ver la cronología).
Registro en Google Sheets con n8n	Probado físicamente	2/11, 14:02: filas escritas en Google Sheets y respuesta de n8n en el panel en unos 2 s.
Ruta óptima entre residuos, pinza para recoger	No existe	El robot va al residuo más cercano con el camino libre y lo empuja.
Sensor de línea del robot	No se usa	Requisito antiguo aplazado a propósito (ver Pendiente_Sensor_Linea.md).
Cámara en Windows o Linux	No existe	El código de cámara usa AVFoundation, que solo hay en Mac. En esos sistemas solo funciona la consola Bluetooth, y sin probar.
Pruebas automáticas	149 pruebas, todas pasan (comprobado el 2/11, unos 20 s)	`work/venv/bin/python -m unittest discover -s tests`. No usan cámara ni robot.

Cronología de las pruebas físicas

Fecha	Qué se probó	Resultado
29/9	Primer receptor cargado por USB	LED, buzzer y respuestas por Bluetooth correctas. Sin movimiento.
26/10	Samsung como cámara, por Camo	Imagen del recuadro a ~30 imágenes por segundo y recuadro elegido solo. Robot sin moverse.
27/10	Firmware V2 cargado. Primeras pruebas con G	El robot llegó al residuo y, en otra prueba, sacó residuos del recuadro y llegó a «Recuadro limpio» varias veces. Fallos: seguimiento perdido al empujar, robot tomado por «caja», robot que quedaba fuera.
27/10	Cámara en la mano	El robot se encontró solo, pero el recuadro no llegó a confirmarse (imagen muy movida).
28/10	3.ª prueba del empuje fuera del recuadro	6 salidas contadas, pero 11 paradas por «el robot salió del recuadro» y latas pisadas. Se cambió: ahora el robot vuelve solo.
29/10	4.ª prueba del empuje	5 salidas contadas (2 cajas, 3 latas), 2 veces «Recuadro limpio», ninguna parada por salir. 8 avisos falsos de «se subió encima» (corregido después).
1/11	Grabación con las hojas y poca luz	Sirvió para corregir dobles detecciones (la hoja roja tomada por «caja»). La prueba de la tarde no llegó a empezar: imagen borrosa y teléfono sin batería.
2/11, 11:32	Primera prueba en vivo de las cinco clases (~15 min)	El robot llevó plástico, lata y caja a la amarilla y el blanqueador a la roja; el panel contó 5. Fallos: Bluetooth cortado al empujar, un blanqueador no contado, vidrio casi nunca visto. Corregidos el conteo y el Bluetooth.
2/11, 12:02	Con esos arreglos	Caja a la amarilla y blanqueador a la roja, contados a la primera.
2/11, 12:24	Ajustes de giro	Fue peor; Juan pidió volver a la versión anterior.
2/11, 12:44-12:48	Primera versión del rumbo por las orugas	«Daba vueltas en su sitio». Se corrigió: nunca dos giros seguidos sin avanzar, y un residuo cuenta como dejado si el 40 % de él pisa la hoja.
2/11, 12:55	Orugas corregidas	Blanqueador a la roja en ~13 s, contado a la primera. Después, una caja pegada a la cinta: el robot salió del recuadro para colocarse detrás, se perdió de vista y se cortó el Bluetooth (problema abierto).
2/11, 13:00	Repetición	Blanqueador a la roja y caja a la amarilla, los dos contados; «Todo clasificado».
2/11, 13:54	Con n8n	Panel con «Automatización n8n: CONECTADO». El blanqueador resbaló junto a la hoja y no se contó; corregido.
2/11, 14:02	Versión actual	Blanqueador a la roja y caja a la amarilla, contados a la primera, apuntados en Google Sheets, respuesta de n8n en el panel en ~2 s.

Resultados medidos

Todas las cifras salen de mediciones reales en el Mac del proyecto. Son escenas pequeñas y de una sola casa: **no son precisiones generales**.

Detector de los cinco residuos (`residuos-propio.pt`, entrenado el 1/11 con 959 fotos):

Clase	Fotos no usadas para aprender (80, misma sesión y escena)	En vivo, 2/11 (% de imágenes en que se vio)
Caja de cartón	100 %	84 %
Lata	82-83 %	97 %
Botella de vidrio	72-88 %	5 %
Botella de plástico	89 %	89 %
Envase de blanqueador	96-99 %	80 %

Las dos cifras de la primera columna dependen de cuánto se exige que la caja dibujada coincida con la real. En esas fotos de prueba no hubo ninguna confusión entre plástico y blanqueador. Ojo: las fotos de prueba son de la misma sesión de fotos que las de aprendizaje (otro momento de la grabación), así que la cifra es optimista. En vivo, el vidrio tumbado se confundía con una lata; **se decidió no seguir con el vidrio**.

Detector del robot (`robot-propio.pt`, 28/10): encontró el robot en 74 de 74 imágenes no vistas y no inventó ninguno en 9 imágenes sin robot, en ~11 ms. El método anterior (Grounding DINO) lo encontraba en 1 de esas 74.

Rumbo por las orugas (2/11): comparado con hacia dónde se movió de verdad el robot en 88 tramos rectos de 9 grabaciones, error mediano 5°, el 81 % por debajo de 15°, y un 10 % de errores grandes (toma la dirección perpendicular), que la navegación descarta. Antes, el rumbo se medía con pasos de prueba y tenía unos 20° de error.

Simulación (no son pruebas físicas):

- Llevar blanqueador y lata a sus hojas (2/11), 60 escenas con ruido y retraso: 56 bien, unos 30 s por escena. Antes de usar las orugas eran 51 bien y 38 s.
- Sacar residuos del recuadro, el modo anterior sin hojas (28/10): 139 de 139 escenas bien en cada uno de tres escenarios de velocidad.

Lo que no hace (todavía)

- No elige una ruta óptima: va al residuo más cercano que tenga el camino libre.
- No recoge: empuja. No tiene pinza.
- No usa el sensor de línea del robot.
- No funciona la cámara en Windows ni Linux.
- No se ha medido el retraso real de la imagen de Camo.
- No se ha comprobado en condiciones de exposición (otra luz, otro suelo, gente alrededor).

Problemas abiertos

Resumen; la explicación y qué hacer están en Problemas.

1. **Residuos pegados a la cinta**: para colocarse detrás, el robot sale del recuadro y puede perderse de vista.
2. **Todavía gira bastante** para colocarse detrás de cada residuo (unos 15 s con la caja).
3. **La botella de vidrio se detecta poco** (decidido no seguir con ella).
4. **El Bluetooth puede cortarse**: hay que pulsar C, E y G.
5. **«Recuadro perdido» desarma el robot** a propósito: hay que pulsar E y G.
6. **Recuadro frágil** si la cámara se mueve o la luz es mala.
7. **La hoja amarilla es pequeña**: un residuo empujado puede sacar a otro; Juan recoge a mano cada residuo en cuanto se cuenta.
8. **Solo funciona en Mac.**

Decisiones que se mantienen

- Siempre una cámara externa; la del Mac, nunca.
- Sin etiqueta impresa en el robot: se reconoce por su aspecto.
- Los LED del robot van apagados durante la visión (su luz confundía el seguimiento).
- A mano, el robot puede acercarse a la cinta y cruzarla; el buzzer avisa al salir. En automático tampoco se para al salir: vuelve solo.
- Se conservan todas las paradas de seguridad (ver Operación).
- El sensor de línea no se activa.

Anexo H. Problemas conocidos

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Instalación · Uso · Estado

Primero, los límites conocidos del prototipo (no son averías: es lo que todavía no está resuelto). Después, soluciones ordenadas por síntoma.

Problemas conocidos del prototipo

Problema	Por qué pasa	Qué hacer
Residuo pegado a la cinta: el robot sale del recuadro, se pierde de vista y a veces se corta el Bluetooth.	Para empujar un residuo hacia su hoja, el robot se coloca detrás de él. Si el residuo está junto a la cinta, en el lado contrario a su hoja, «detrás» queda fuera del recuadro, a veces fuera de la imagen.	Coloca los residuos al menos a un robot de distancia de la cinta. Si pasa: Espacio, vuelve a poner el robot dentro y pulsa C (si se cortó), E y G.
El robot gira bastante antes de empujar.	Colocarse detrás de cada residuo necesita varias maniobras (con la caja, unos 15 s). Ya no da vueltas en su sitio, pero sigue maniobrando.	Es normal en el prototipo. Explícalo en la exposición.
La botella de vidrio casi no se detecta (5 % de las imágenes en vivo).	Es pequeña; tumbada, el detector la confunde con una lata.	Se decidió no seguir con ella: mejor no usarla en la demostración.
Se corta el Bluetooth («Bluetooth detenido: …»).	Si tres confirmaciones seguidas del robot tardan más de 0,45 s, el programa cierra la conexión por seguridad. Pasa más si el robot se aleja o queda tapado.	Pulsa **C**, **E** y **G**. Mantén el Mac cerca del recuadro.
«Recuadro perdido o imagen atrasada» y el robot queda deshabilitado.	Sin una imagen reciente del recuadro, el robot no sabe dónde está. Se deshabilita a propósito.	Cuando vuelva el recuadro verde: **E** y **G**.
El recuadro no se confirma o se pierde con la cámara en la mano, poca luz o sombras.	El detector de cinta necesita los cuatro lados nítidos y suelo claro a ambos lados de la cinta.	Cámara fija, luz uniforme, cinta sin tapar.
Un residuo empujado saca a otro de la hoja amarilla.	La hoja es pequeña.	Retira a mano cada residuo en cuanto el panel lo cuente.
Solo funciona en Mac.	La cámara se lee con AVFoundation, que solo existe en macOS.	En Windows o Linux solo funciona la consola Bluetooth.

Instalación

No encuentra Python o una biblioteca

Usa siempre el Python del proyecto (`work/venv/bin/python`), no otro del sistema. Si aparece `No module named …`, vuelve a instalar las bibliotecas con ese mismo Python (paso 3 de Instalación). No copies la carpeta `work/venv` de otro ordenador: créala de nuevo.

OpenCV no encuentra CSRT

`requirements_vision.txt` instala `opencv-python` y `opencv-contrib-python`, que comparten el mismo módulo y a veces se pisan. Deja solo la variante contrib:

```
work/venv/bin/python -m pip uninstall -y opencv-python opencv-contrib-python opencv-python-he
adless opencv-contrib-python-headless
work/venv/bin/python -m pip install opencv-contrib-python==5.0.0.93
work/venv/bin/python -c "import cv2; print(hasattr(cv2, 'TrackerCSRT_create'))"
```

Debe imprimir `True`. No uses las variantes `headless`: no abren ventanas.

Los modelos no aparecen al clonar

Es intencionado: los modelos no se suben a GitHub. Los generales se descargan con `work/venv/bin/python outputs/preparar_modelos.py` (necesita Internet, Git y curl). Los propios (`residuos-propio.pt`, `robot-propio.pt`) hay que pedírselos a Juan. Si una descarga se corta, repite el preparador; si un modelo quedó dañado, apártalo y vuelve a descargarlo.

La ventana dice «Detector no disponible: …»

Falta un modelo general (`residuos-world-small.pt` u otro): sin él no arranca ningún detector, tampoco los propios. Ejecuta `preparar_modelos.py`. El mensaje completo dice qué archivo falta.

Cámara

No encuentra la cámara

1. Ejecuta `work/venv/bin/python outputs/vision_linea_robot.py --listar-camaras`.
2. Si el teléfono no aparece: abre Camo Studio y Camo, comprueba que la extensión de cámara de Camo está activada (Ajustes del Sistema → General → Ítems de inicio y extensiones → Extensiones de cámara) y que el cable es de datos.
3. Si aparecen varias cámaras reconocidas, elige una con `--camara camo` (o `samsung`, `lifecam`, o su número).
4. Cierra otros programas que usen la cámara (videollamadas, otro programa del proyecto).
5. Revisa en Ajustes del Sistema → Privacidad y seguridad → Cámara que Terminal tiene permiso.

La cámara del Mac se rechaza siempre, a propósito.

Imagen negra con el logo de Camo, o el programa se cierra

Camo perdió el teléfono (desconectado, bloqueado o sin batería). Vuelve a conectarlo y abre Camo. Con el robot en marcha, si la imagen no cambia en 0,6 s, el programa lo para. Ten el teléfono cargando durante la demostración: una prueba del 1/11 no llegó a empezar porque se quedó sin batería.

Aviso de bandas negras o de imagen vertical

Bandas negras: la proporción pedida no coincide con la del teléfono; prueba `--resolucion 1280x720` o `--resolucion 1920x1080` (las bandas podrían confundirse con cinta). Imagen vertical: gira el teléfono a horizontal.

Recuadro y hojas

El recuadro no aparece, o dice «Borde conservado»

- Deben verse los **cuatro lados** y las esquinas, con suelo claro a ambos lados de la cinta y una franja exterior.
- Que no haya objetos grandes sobre la cinta. El robot sí puede taparla un poco.
- Evita sombras fuertes y reflejos en la cinta.
- Pulsa `t` para buscarlo de nuevo (sobre todo si se movió la cámara).
- Si se ve el candidato en amarillo pero no se confirma, `b` lo confirma a mano.

Las hojas no aparecen

- Tienen que estar **dentro del recuadro**, planas y con un color saturado. La roja tiene que verse **casi tan clara como el suelo o más** (la caja roja es más oscura y así no se confunden).
- Necesitan verse en el mismo sitio alrededor de un segundo para confirmarse. Después se recuerdan aunque algo las tape.
- Si las moviste, pulsa `t` (borra recuadro y hojas y los busca de nuevo).
- Sin la hoja de un residuo, el robot lo saca del recuadro en vez de llevarlo a la hoja.

La hoja roja aparece como «caja»

Con poca luz, la hoja roja y la caja roja tienen casi el mismo color. El programa descarta la detección que coincide con la hoja si no hay nada más oscuro encima. Si sigue pasando, mejora la luz.

Robot

No encuentra el robot (no aparece la caja azul «RANGER»)

- Comprueba que `outputs/models/robot-propio.pt` existe y que la línea 7 de la ventana termina en «| robot».
- El robot debe estar dentro del recuadro o junto a él, con las orugas visibles.
- Respaldo: pulsa `r`, arrastra un rectángulo que incluya todo el robot y pulsa Enter.

La caja azul se va a otro objeto

Al empujar, el seguimiento puede quedarse en el residuo. El detector del robot lo corrige solo en cuanto lo ve con suficiente confianza. Si no se corrige, pulsa `n` (reiniciar todo) o `r`.

El Bluetooth no encuentra el robot

1. Enciende el robot y comprueba que el módulo Bluetooth está puesto.
2. Activa el Bluetooth del Mac y da permiso de Bluetooth a Terminal.
3. Cierra mBlock, la aplicación del teléfono u otros programas conectados al robot.
4. Ejecuta `work/venv/bin/python outputs/ranger_bluetooth.py --listar`. Si el nombre no es `Makeblock_LE10a56219eb91`, pásalo con `--nombre`.

Conecta pero no responde

El programa espera la versión `RANGER_READY_V2_20261019`. Si el robot tiene otro programa (por ejemplo, el de fábrica), carga el firmware con `outputs/Actualizar_Ranger.command` (Operación). Prueba primero sin movimiento: `--comando x` y `--comando p`.

No se mueve aunque pulse las teclas

Comprueba, en este orden:

1. La ventana del vídeo está activa (haz clic en ella) y las teclas en minúscula.
2. Recuadro verde.
3. Caja azul sobre el robot.
4. «Bluetooth conectado» (si no, **C**).
5. «Movimiento: HABILITADO» (si no, **E**).
6. Robot con batería y orugas libres.

Después de Espacio, `t`, `n`, `b`, `r` o de cualquier parada de seguridad hay que volver a pulsar **E** (y **G** para el automático).

Se para con «el robot no avanza»

Varios pasos hacia delante sin moverse: batería baja, orugas atascadas, o la caja azul no está sobre el robot. Revísalo y pulsa **E** y **G**.

Se sube encima de un residuo

Pasa sobre todo con latas tumbadas. El robot lo detecta (el residuo aparece detrás de él), retrocede, gira unos 70° y lo intenta desde otro ángulo. Si falla tres veces, lo deja para más tarde.

Detección

Un residuo no se detecta

- Comprueba que es uno de los cinco que conoce el detector y que la línea 7 de la ventana dice «modelo propio».
- Acerca la cámara (sin dejar de ver todo el recuadro): en la vista de trabajo los residuos miden solo 20-40 px.
- Mejora la luz; evita reflejos sobre la lata.
- No amontones residuos ni los tengas en la mano.
- Para mirarlo de cerca, usa el zoom (`+`, clic sobre el objeto): no cambia la detección, pero ayuda a ver qué pasa.

Confunde la botella de plástico con el blanqueador

Si duda entre los dos, el programa elige **blanqueador** a propósito: es peor mandar un químico peligroso a la hoja de reciclables. Una botella de plástico en la hoja roja es el error «seguro».

Aparecen «objeto por identificar» en amarillo

Solo con los detectores generales encendidos: con `--otros-residuos`, o si falta `residuos-propio.pt`. No se cuentan ni se empujan.

Conteo y panel

Un residuo llegó a su hoja y no se contó

- Tiene que haberse visto **fuera de su hoja antes**: un residuo que empieza ya sobre su hoja no cuenta.
- Tiene que quedar con el centro encima de la hoja o con al menos el 40 % de su caja pisándola.

- Si el robot lo tapa, se confirma cuando retrocede y vuelve a verse.
- Reglas completas: Operación.

Se contó dos veces

Pasa si alguien lo quita de la hoja, lo deja fuera más de 2 s y lo vuelve a poner: es a propósito. Si fue sin tocarlo, guarda la sesión con `--diagnostico` y revisa `estado.json` (lista `tracks`). Para empezar de cero, cierra y abre el programa: la sesión nueva empieza en 0 y las anteriores se conservan.

El panel dice «Sin imagen de la cámara» o «Panel desconectado»

- El programa principal está cerrado, o el panel se abrió con `Abrir_Dashboard.command` (ese no tiene cámara).
- Abre `Probar_Robot_Linea.command` y recarga http://127.0.0.1:8765.
- Si al arrancar sale «Dashboard: puerto ocupado», ya hay otro panel abierto en el 8765: ciérralo, o usa `--dashboard-port 8766`.

«Automatización n8n: SIN RESPUESTA» o «PROGRAMA CERRADO»

- SIN RESPUESTA: n8n no está abierto o el flujo no está publicado. Abre `outputs/n8n/Abrir_n8n.command`. Los residuos pendientes se envían solos cuando n8n responde, mientras el programa siga abierto.
- PROGRAMA CERRADO: el programa de la cámara no está abierto.
- APAGADO: se arrancó con `--sin-n8n`.
- Configuración y errores de Google Sheets: N8N.md.

GitHub

GitHub rechaza archivos grandes

Comprueba que `.gitignore` está presente y que no estás añadiendo a la fuerza modelos, `work/` o capturas. Si un archivo grande ya está en un commit, añadirlo a `.gitignore` no lo quita del historial. No borres los modelos locales para arreglarlo: pide ayuda antes de limpiar el historial.

Qué incluir al pedir ayuda

- Qué hacías, el comando exacto o el lanzador y el mensaje completo.
- Las 9 líneas de texto de la ventana (una captura sirve).
- Si es posible, una carpeta de diagnóstico: `--diagnostico work/diagnostico` guarda `camara.jpg`, `vista.jpg` y `estado.json` cada segundo.

Revisa las imágenes antes de compartirlas: muestran la casa real.

Anexo I. Operación V2

Actualizado: 2 de noviembre de 2026

¿Empiezas desde cero? Lee primero la Guía de estudio.

Inicio · Uso · Robot y Bluetooth · Estado

El Manual de uso dice **qué teclas pulsar**. Este documento explica **qué hace el sistema por dentro** cuando las pulsas: el programa de la placa del robot, las reglas de seguridad, cómo decide el robot en el modo automático y cuándo se cuenta un residuo. El nombre del archivo viene de la «versión 2» del firmware (19 de octubre), con la que empezó este modo de trabajo.

Firmware V2: qué es y cómo se carga

El **firmware** es el programa que vive en la placa Auriga del mBot Ranger. El nuestro (`outputs/ranger_bluetooth.ino`) obedece letras que llegan por Bluetooth:

Letra	Qué hace
`w s a d`	Adelante, atrás, girar a la izquierda, girar a la derecha (potencia fija).
`x`	Parar los motores.
`p`	Pitido corto **sin parar los motores**.
`0 1 2 3`	LED apagados, rojos, verdes o azules.
`?`	Responde su versión: `RANGER_READY_V2_20261019`.

Si pasan **600 ms sin una letra de movimiento**, la placa para los motores por su cuenta (y responde `AUTO_STOP`). Así, si el Mac se cuelga o se corta el Bluetooth, el robot no sigue andando. Arranca con los LED apagados y sin pitar. No lee el sensor de línea.

Estado: cargado en el robot el 27/10 (sustituyó, con permiso de Juan, un programa de «control por aplausos» cuyo código no estaba en el Mac). Solo hay que volver a cargarlo si alguien restaura el programa de fábrica o carga otro con mBlock.

Para cargarlo:

1. Cierra mBlock y cualquier programa que use el robot.
2. Abre `outputs/Actualizar_Ranger.command`. Espera a que pida el cable.
3. Conecta el robot al Mac por USB. El programa reconoce la placa, compila el firmware, lo carga y comprueba la versión.
4. No desconectes el cable hasta leer `Actualización verificada: RANGER_READY_V2_20261019`.

Desde Terminal es lo mismo:

```
work/venv/bin/python outputs/actualizar_ranger.py --esperar
```

- Con varios puertos USB, indica el del robot: `--puerto /dev/cu.usbserial-1120` (es el que tenía en el Mac de Juan; compruébalo).
- Si la placa no responde como nuestro receptor (programa de fábrica u otro), la carga automática se niega. Tras comprobar que el puerto es el del robot: `--puerto ... --carga-inicial`.
- Tras abrir el puerto USB, la placa tarda unos 3 s en arrancar.

- La compilación usa las herramientas de la placa que instala **mBlock** en el Mac (`--toolchain RUTA` para otras). No está probada en Windows ni Linux.
- **Por Bluetooth no se puede cargar el firmware.** Si el programa encuentra una versión antigua por Bluetooth, pide el cable en vez de mover el robot.

Seguridad: cuándo se para el robot

El robot se mueve a **pasos cortos**: cada orden dura como mucho unas décimas de segundo y hay que renovarla. Si algo falla, basta con dejar de enviar órdenes. Estas son todas las paradas:

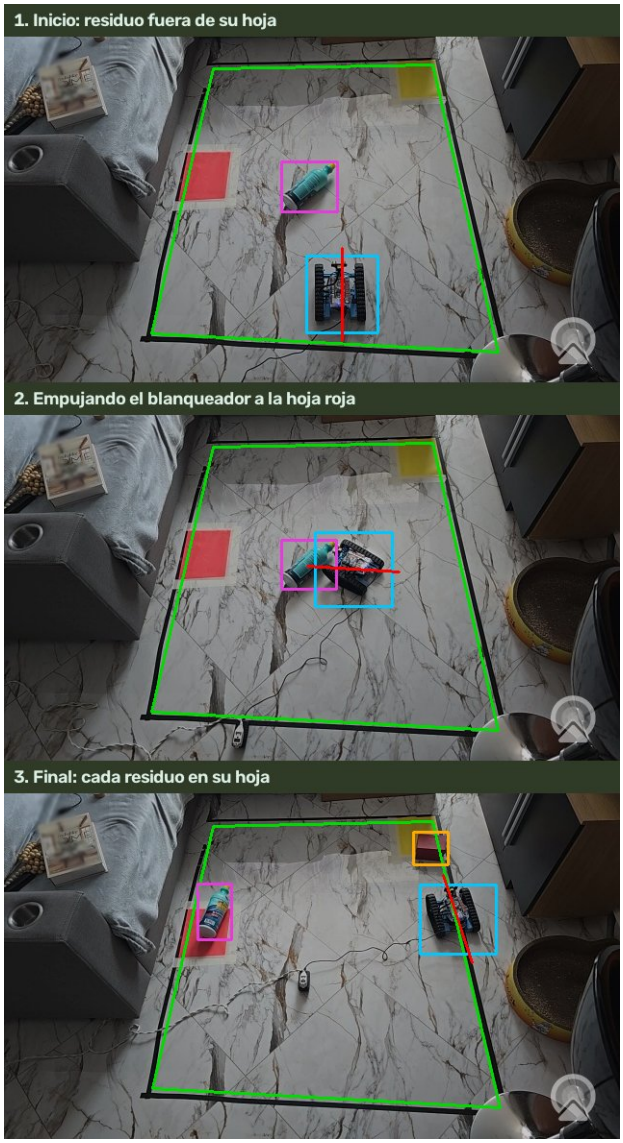
Situación	Qué pasa	Para seguir
Espacio o `x`	Para y deshabilita el movimiento. Apaga el modo automático.	`E` y `G`
Recuadro perdido o no visible («Borde conservado»), o imagen con más de 0,3 s de retraso	Para y **deshabilita** (a propósito: sin una imagen reciente del recuadro, el robot no sabe dónde está). Mensaje: «Recuadro perdido o imagen atrasada».	`E` y `G`
Seguimiento del robot perdido	Para. El detector lo busca de nuevo.	`E` y `G` cuando vuelva la caja azul
Imagen congelada 0,6 s con el robot en marcha (Camo sin teléfono)	Para y deshabilita.	Revisa la cámara; `E` y `G`
Bluetooth: 3 confirmaciones seguidas tardan más de 0,45 s	Cierra la conexión (una o dos lentas se toleran).	`C`, `E` y `G`
Cualquier tecla `t`, `n`, `b`, `r` o un paso manual	Para el automático (y `t`, `n`, `b`, `r` deshabilitan).	`E` y `G`
La placa no recibe letras en 600 ms	El firmware para los motores.	Automático al llegar nuevas órdenes.

Lo que **no** para al robot, por decisión del proyecto:

- **Acercarse a la cinta o cruzarla.** A mano se permite. Cuando el robot sale del recuadro (su caja pasa la cinta más de 5 px durante 0,25 s), el buzzer pita cada 2 s, sin parar los motores.
- **Salir del recuadro en automático.** El robot vuelve solo hacia el centro (marcha atrás si mira hacia fuera). Antes se paraba y había que ir a buscarlo.
- **El sensor de línea** de la placa no se usa (requisito aplazado).

Los LED del robot se apagan al conectar: su luz confundía el seguimiento.

El modo automático (tecla G)



Secuencia: residuo fuera de su hoja, empuje y final

El código está en `outputs/navegacion.py`. En cada imagen decide la siguiente orden corta; el programa principal la envía con las mismas protecciones que un paso manual. Todas las posiciones se miden en **coordenadas del recuadro** (el recuadro «aplanado» a un cuadrado de 400 × 400), así que un pequeño movimiento de la cámara no lo confunde.

Qué residuo va a cada hoja

Residuo	Hoja	Grupo
Caja de cartón, lata, botella de vidrio, botella de plástico	**Amarilla**	Aprovechables
Envase de blanqueador	**Roja**	Peligrosos

Si el detector duda entre botella de plástico y blanqueador, gana el blanqueador: es peor un químico en la hoja de reciclables que una botella de agua en la de peligrosos.

Los pasos

1. **Elegir residuo.** El más cercano dentro del recuadro que no esté ya en su hoja, prefiriendo los que tienen libre el camino hasta su hoja.

2. **Saber hacia dónde mira el robot.** La cámara ve el robot como un rectángulo, sin frente. El programa usa la dirección de sus **orugas** en la imagen (Arquitectura) y, para saber cuál es el frente, lo que se desplaza tras cada paso hacia delante. También aprende cuántos grados gira por segundo. Nunca encadena dos giros sin avanzar entre ellos (así dejó de dar vueltas en su sitio).
3. **Colocarse detrás.** Va a un punto detrás del residuo, en la línea que une el residuo con el centro de su hoja. Si el camino pasa por encima del residuo, lo rodea por un lado.
4. **Empujar.** Avanza hacia el centro de la hoja con el residuo delante, en pasos de 0,15 a 0,4 s, corrigiendo el rumbo. Si el residuo resbala hacia un lado, vuelve a colocarse detrás.
5. **Dejarlo.** Cuando el residuo está sobre su hoja, retrocede un poco y pita. Si el robot lo tapa y su morro ya llegó al centro de la hoja, solo lo da por dejado si la última vez que lo vio estaba junto a la hoja; si no, retrocede para volver a verlo.
6. **Siguiente.** Repite con el resto. Antes de terminar retrocede un paso por si tapaba algún residuo y, si no queda ninguno, dice «Todo clasificado: cada residuo en su hoja» y pita.

Situaciones especiales

- **Se sube a un residuo o se atasca** (el residuo vuelve a verse detrás del robot, o 3 pasos sin avanzar): retrocede 0,45 s, gira unos 70° (alternando el lado), avanza 0,3 s y vuelve a intentarlo desde otro ángulo.
- **Tres intentos o 120 s con el mismo residuo**: lo deja 60 s y sigue con otros.
- **Si sigue sin poder avanzar** después de varios intentos: se detiene con «el robot no avanza (revisa baterias, orugas o seguimiento)».
- **Residuo sin hoja** (solo pasa con `--otros-residuos`, con clases como papel, o si no se ha reconocido su hoja): lo saca del recuadro. Va recto hacia él, lo empuja por esa misma línea hasta que pasa la cinta, retrocede y sigue. Al final dice «Recuadro limpio». Este fue el primer modo del proyecto (probado físicamente del 27 al 29/10).

Límites conocidos

- Con un residuo pegado a la cinta, en el lado contrario a su hoja, el punto «detrás» queda fuera del recuadro: el robot sale, puede perderse de vista y cortarse el Bluetooth.
- Colocarse detrás requiere varias maniobras: con la caja, unos 15 s.
- No calcula una ruta óptima: va de uno en uno, al más cercano.

Ver Problemas.

Cuándo cuenta un residuo

El conteo está en `outputs/conteo_residuos.py`. Sigue a cada residuo en coordenadas del recuadro, así que no importa que la cámara se mueva un poco ni que el robot lo tape un momento al empujarlo (las trayectorias se recuerdan 60 s).

Un residuo cuenta como **separado** cuando:

1. Se vio **fuera de su hoja** (confirmado en al menos 2 detecciones durante 0,35 s), y después
2. Quedó **sobre su hoja**: su centro encima, **o al menos el 40 % de su caja** pisando la hoja, confirmado igual. También vale verlo encima y que luego el robot lo tape durante 1 s sin que aparezca en otro sitio.

No cuenta:

- Un residuo que **ya estaba en su hoja** desde el principio.

- Un residuo en la **hoja equivocada**.
- Un residuo que **sale del recuadro** (si tiene hoja).
- Volver a empujar uno que ya se contó.

Cuenta otra vez si alguien lo **quita de su hoja**, sigue fuera al menos 2 s y lo vuelve a dejar. Excepción: si se ve fuera de la hoja en los 5 s siguientes a contarlo, no había llegado de verdad (lo tapaba el robot) y la siguiente llegada no suma.

Da igual quién lo lleve: el robot en automático, el robot con las teclas o una mano. El conteo es una **estimación visual**: no comprueba el material ni certifica el reciclaje.

Para residuos **sin hoja** (solo con `--otros-residuos`) se usa la regla antigua: cuenta al pasar de dentro del recuadro a fuera (centro más allá de la cinta y como mucho una cuarta parte del residuo aún dentro), también si sale por un lado de la imagen o si el robot avisa de que lo sacó y no vuelve a verse dentro.

Cada residuo contado se guarda en `work/estadisticas.sqlite3` (sesión, número de trayectoria, tipo, hora y confianza), se muestra en el panel y se envía a n8n.

El panel web

`outputs/dashboard.py` es un pequeño servidor que **solo escucha en el propio Mac** (127.0.0.1, puerto 8765). Lo abre el programa principal; `Abrir_Dashboard.command` lo abre solo, sin cámara. Si el puerto ya está ocupado (por ejemplo, por `Abrir_Dashboard.command`), el programa sigue funcionando y ese otro panel muestra sus datos, porque leen la misma base de datos, pero sin la imagen en vivo.

Dirección	Qué devuelve
`/`	La página del panel.
`/api/estado`	Los datos en JSON (sesiones, eventos, estado en vivo, n8n).
`/video.mjpg`	La cámara en vivo con las marcas. Solo se codifica mientras alguien la mira (hasta 10 imágenes por segundo).
`/foto.jpg`	La última imagen (si tiene menos de 3 s).
`/exportar.csv`	Los residuos separados de una sesión: sesión, objeto, categoría, fecha UTC y confianza.

No tiene botones para mover el robot ni publica nada en Internet. No inventa datos: sin residuos contados, muestra ceros. Qué se ve en la página: Uso.

n8n

Cada residuo contado se envía, en un hilo aparte, a `http://localhost:5678/webhook/residuos` (cambiable con `--n8n URL`; `--sin-n8n` lo apaga). El mensaje lleva fecha, hora, residuo, clase, hoja, grupo, confianza, sesión y total de la sesión; nunca imágenes. n8n lo apunta en Google Sheets y responde un mensaje, que el panel muestra en «Automatización n8n».

- Si n8n no responde, los avisos esperan en orden (hasta 500) y se reintentan cada 5 s mientras el programa siga abierto.
- Si n8n responde con un error, se intenta 3 veces y se avisa en el panel (no se reintenta sin fin para no duplicar filas).
- Nada de esto frena al robot.

La instalación y el flujo se explican en N8N.md.

Comprobaciones antes de una demostración

1. Pruebas automáticas (no necesitan cámara ni robot; tardan unos 20 s):

```
work/venv/bin/python -m unittest discover -s tests
```

Deben terminar con `OK` (149 pruebas).

1. Robot en el suelo, con batería, sin cables tirantes y con alguien preparado para pulsar Espacio.
2. Bluetooth sin movimiento: `work/venv/bin/python outputs/ranger_bluetooth.py --comando p` debe pitar.
3. Con `Probar_Robot_Linea.command`: recuadro verde, hojas, robot y residuos reconocidos; panel con la cámara en vivo.
4. Un paso a mano (`w`) y luego `G` con un solo residuo. Comprueba el «+1» en el panel y, si se usa, la fila en Google Sheets.

Las pruebas automáticas y la simulación no sustituyen a esta comprobación en el lugar de la exposición: allí la luz, el suelo y el montaje de la cámara serán distintos.